S4

Objectius

- Optimitzar taules grans amb Particionament i Clustering.
- Utilitzar Vistes Materialitzades per a Smart Caching.
- Dominar SQL Analític (CTEs, Window Functions).
- Treballar amb dades niades (Nested Data).

Lliurament:

Revisa el material a entregar en cada exercici.

- **Codi i Infraestructura (.sql) Els arxius .sql amb tots els scripts generats.**
- **Evidència d'Execució i Rendiment (PDF)** Un document PDF únic que contingui les captures de pantalla de la Consola de **BigQuery** per a cada exercici.
  - **Requisit Crític**: Per validar que la teva optimització funciona, la captura ha de mostrar clarament:
    - El **codi SQL** a l'editor.
    - El missatge de validació (check verd ✅) o la pestanya *Job Information* on es vegin els **Bytes processats** (Bytes processed).

Recorda revisar les millors pràctiques abans de fer el lliurament.

Com a Analytics Engineer, la teva responsabilitat no és només lliurar dades, sinó assegurar l'escalabilitat i l'eficiència de la plataforma.

En aquest sprint final, redissenyaràs l'arquitectura de les dades per optimitzar costos (**Partitioning**), introduiràs lògica analítica avançada (**Window Functions**) i modelaràs estructures complexes (**Arrays**) per tancar el cicle amb l'automatització.

⚠️ **Important**

- **Control de Costos**: A BigQuery, cada consulta costa diners. Abans d'executar res, fixa't sempre en el missatge del validador ("Dry Run") a dalt a la dreta.
- **Ubicació de les Dades**: Assegura't que el teu Dataset estigui a la regió EU (Multiple regions) o les connexions fallaran.

Limitacions del Sandbox

La zona de proves de BigQuery (Sandbox) està subjecta als següents límits:

- S'apliquen tots els límits i quotes habituals de BigQuery.
- Tens 10 GB d'emmagatzematge actiu i 1 TB de dades de consulta processades cada mes sense cap cost.

- ⚠️ **Caducitat**: Totes les taules, vistes i particions caduquen de forma automàtica després de **60 dies**.
- No admet funcions avançades com: Transmissió de dades (Streaming), DML (`UPDATE` / `DELETE` massius) o el Servei de transferència de dades.

Nivell 1: Entorn i Ingesta Híbrida (Code-First)

> ### Escenari:
>
> El quadre de comandament financer tarda massa a carregar i el cost és inacceptablement alt. S'ha identificat que el problema crític són els informes que creuen vendes amb regions geogràfiques.

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

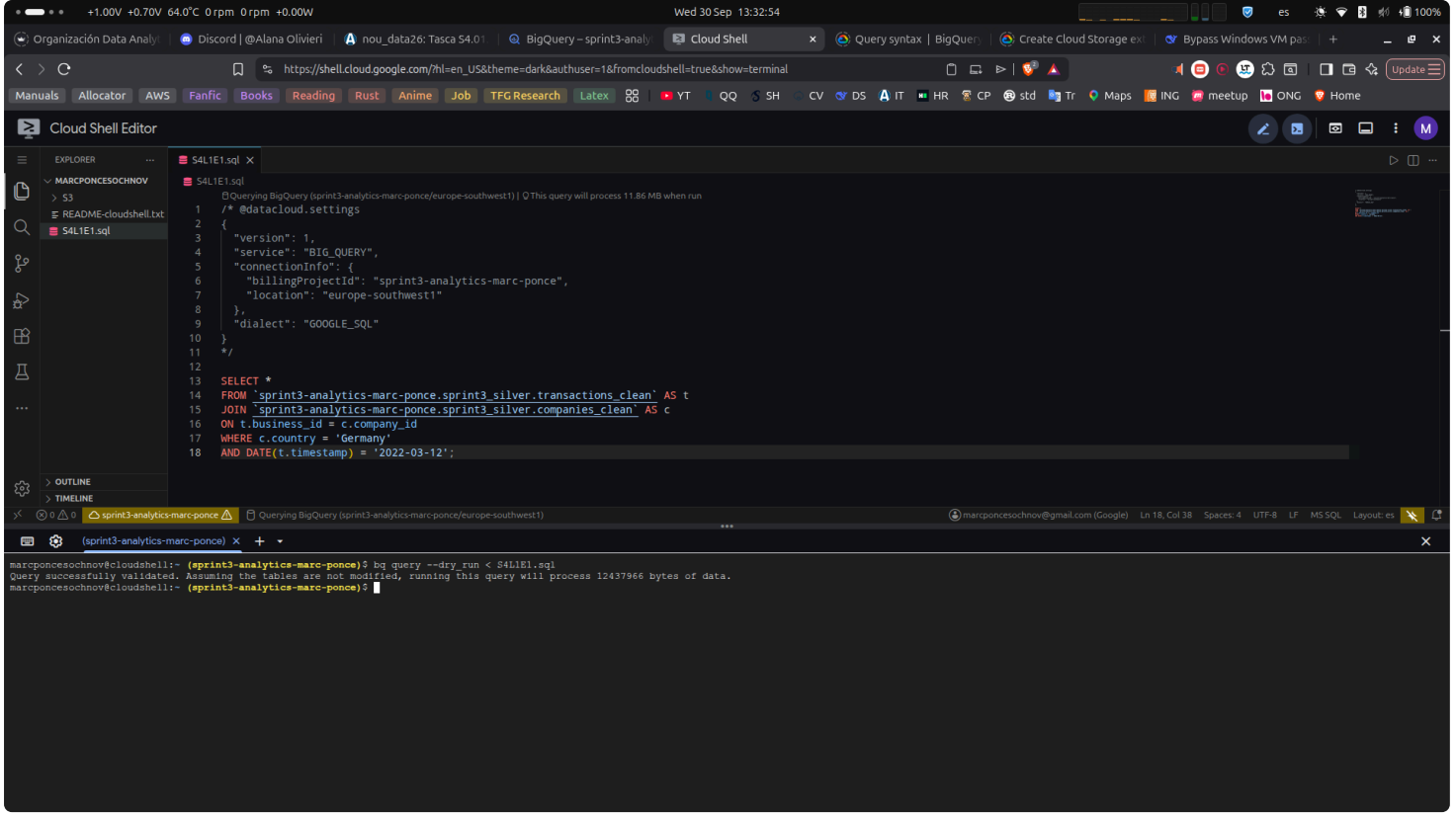
El Country Manager d'**Alemanya** necessita revisar urgentment les transaccions del dia **12 de març de 2022**.

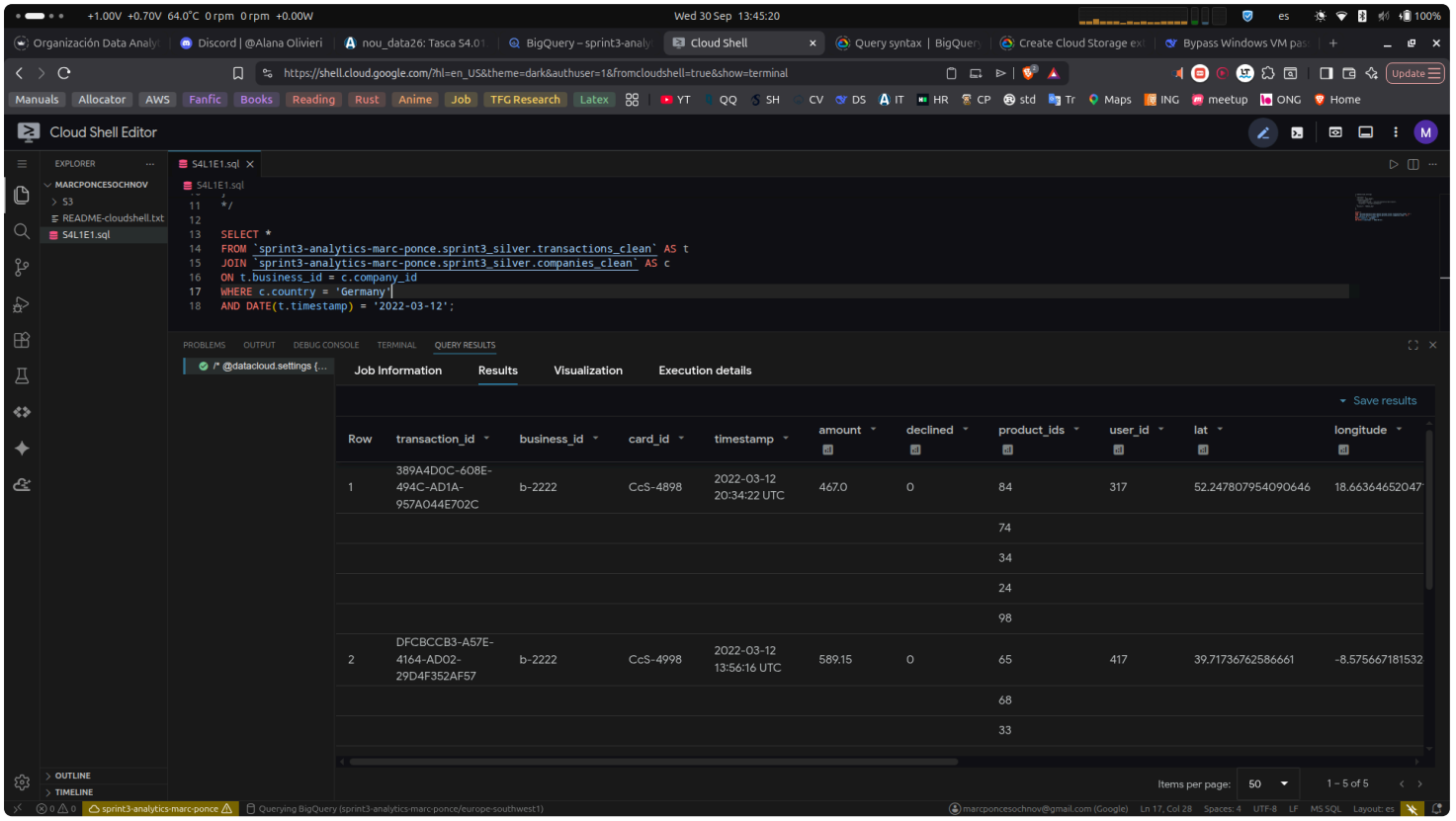
🧩 Tasca:

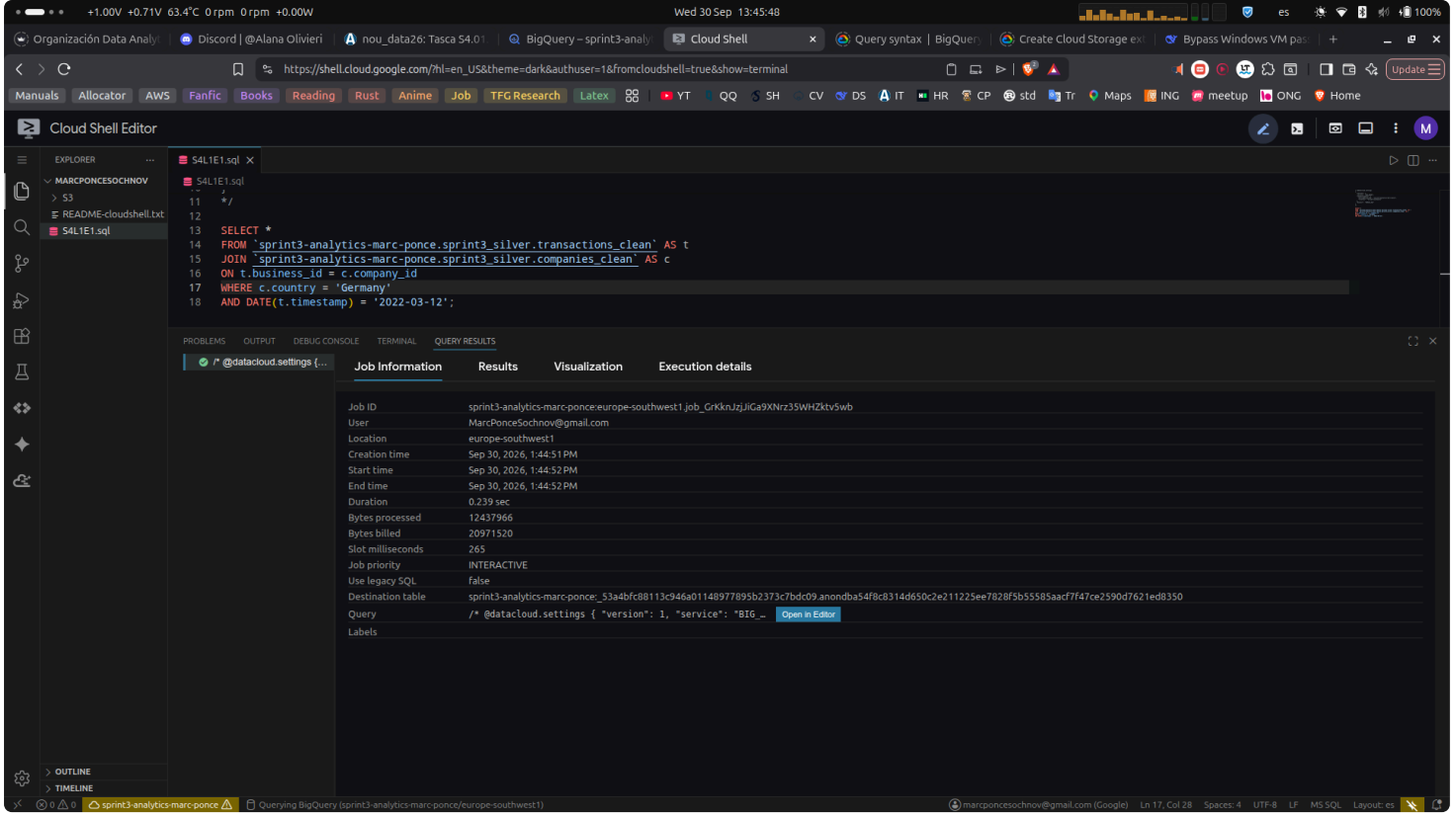
1. Escriu la consulta que uneix (`JOIN`) transaccions i companyies.
2. Filtra els resultats per la data indicada i el país `"Germany"` .
3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).
   👀 *Observació: Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).*

» The dry-run estimation can be seen both at the top of the code `11.86MB` and as an explicit command that yields an estimation of `12437966 bytes` i.e. `12.43MB` . The difference is in the calculation since a `MB` can be either `1048576 bytes` counting by the binary system or `10000000 bytes` in the decimal system: `12437966/(1024*1024)= 11.8617687225` . Clearly, Google defaults to the binary system for its displays.

```sql
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.companies_clean` AS c
ON t.business_id = c.company_id
WHERE c.country = 'Germany'
AND DATE(t.timestamp) = '2022-03-12';
```


Cloud Shell Editor
S4L1E1.sql
/* @datacloud.settings
{
"version": 1,
"service": "BIG_QUERY",
"connectionInfo": {
"billingProjectId": "sprint3-analytics-marc-ponce",
"location": "europe-southwest1"
},
"dialect": "GOOGLE_SQL"
}
*/
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.companies_clean` AS c
ON t.business_id = c.company_id
WHERE c.country = 'Germany'
AND DATE(t.timestamp) = '2022-03-12';
marcponcesochnov@cloudshell:~ (sprint3-analytics-marc-ponce)$ bq query --dry_run < S4L1E1.sql
Query successfully validated. Assuming the tables are not modified, running this query will process 12437966 bytes of data.
marcponcesochnov@cloudshell:~ (sprint3-analytics-marc-ponce)$


Cloud Shell Editor
Job Information
Results
Visualization
Execution details
Save results
Row
transaction_id
business_id
card_id
timestamp
amount
declined
product_ids
user_id
lat
longitude
1
389A4D0C-608E-494C-AD1A-957A044E702C
b-2222
CcS-4898
2022-03-12 20:34:22 UTC
467.0
0
84
317
52.247807954090646
18.66364652047
74
34
24
98
2
DFCBCCB3-A57E-4164-AD02-29D4F352AF57
b-2222
CcS-4998
2022-03-12 13:56:16 UTC
589.15
0
65
417
39.71736762586661
-8.575667181532
68
33
Items per page:
50
1 – 5 of 5



» The `bytes processed` coincides exactly with the estimation of the dry run. On the other hand we were billed exactly `20MB` in the binary system: `20971520/(1024*1024)=20`. This happens because there's a minimum `10MB` bill per table referenced in a query. Since here we refer to two tables, we are billed two references.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

> ### Escenari:
>
> Després del diagnòstic de l'exercici anterior, hem confirmat que filtrar la taula de transaccions per data o per empresa provoca un *Full Table Scan* (lectura completa de la taula), cosa que és ineficient. Per solucionar-ho a la capa de consum (*Gold*), materialitzarem una versió optimitzada de la taula de fets.
>
> **Problema del sandbox:** com que utilitzem un entorn gratuït sense targeta de crèdit, BigQuery esborra automàticament les dades de les particions amb més de 60 dies d'antiguitat. Atès que les nostres dades històriques són antigues, si particionem directament la taula, aquesta acabarà quedant buida. Per evitar-ho, ho resoldrem en dos passos: primer, actualitzarem automàticament les dates de les dades i, després, aplicarem l'optimització física.

🧩 Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el `timestamp` original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

Pista Tècnica: Per fer-ho sense escriure totes les columnes a mà, pots utilitzar: `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions: `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

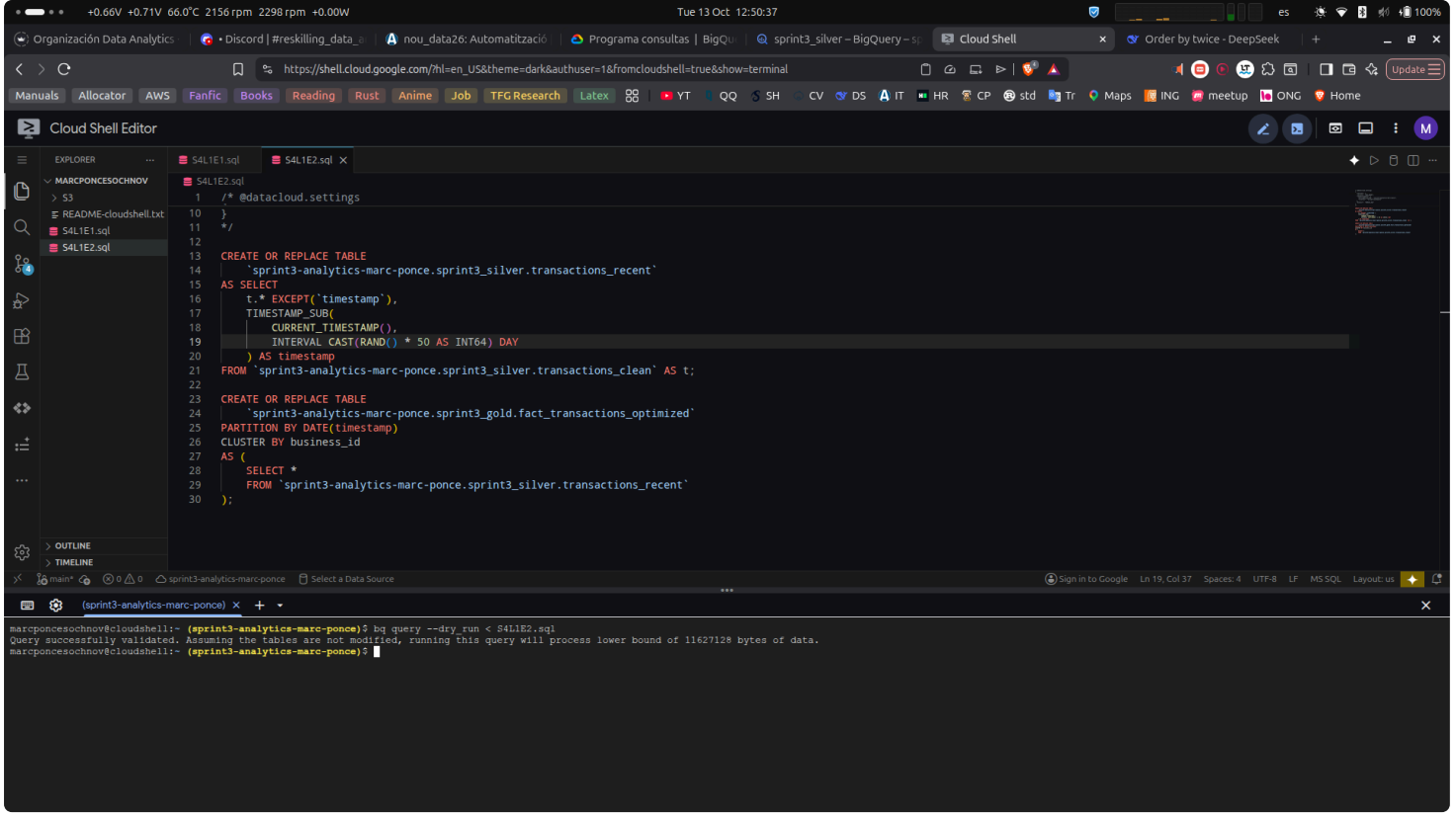
- **Particionament (Partitioning)**: Divideix la taula per la data del camp `DATE(timestamp)`. Això permetrà que les consultes que filtrin per dia (`WHERE date = ...`) només llegeixin la partició necessària.
- **Clustering**: Ordena les dades dins de cada partició per `business_id`. Això accelerarà dràsticament els filtres per companyia i els encreuaments (`JOIN`s) amb la dimensió d'empreses.

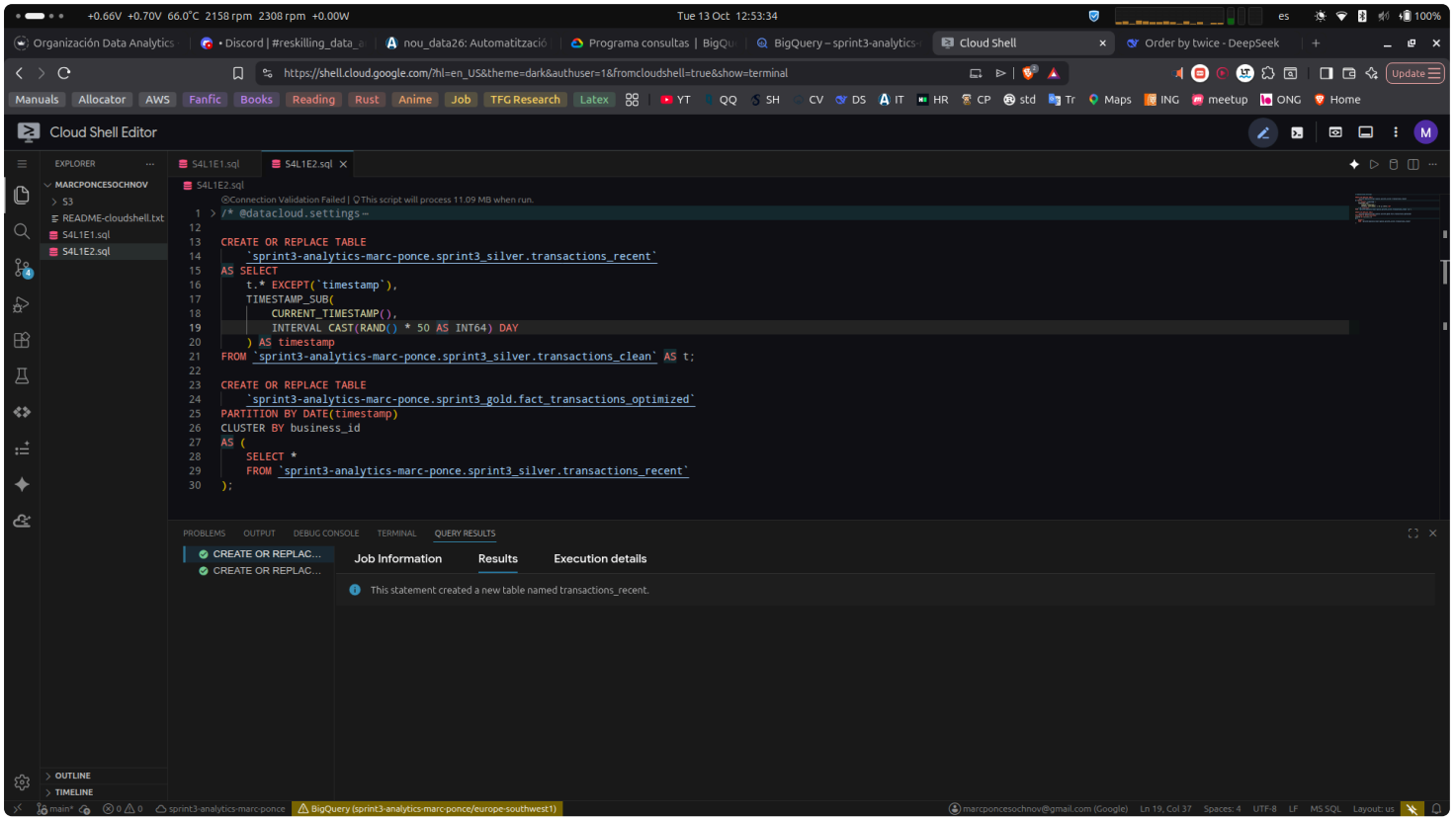
👉 Entrega:

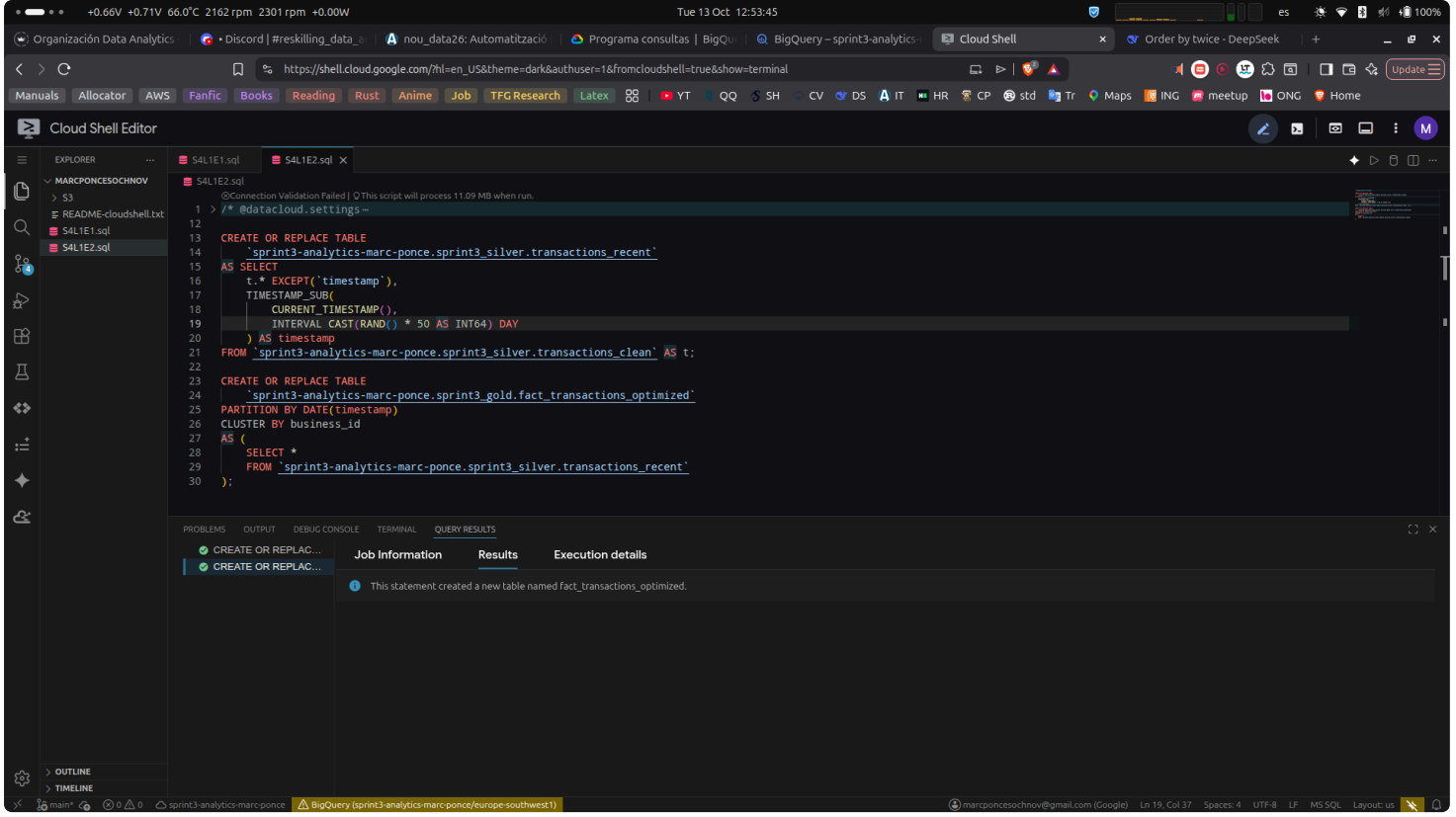
- El codi SQL utilitzat per al Pas 1.
- El codi DDL (`CREATE OR REPLACE TABLE...`) utilitzat al Pas 2.
- Una captura de pantalla dels "Detalls" de la nova taula `fact_transactions_optimized` on es pugui comprovar que està particionada i agrupada en clústers correctament.

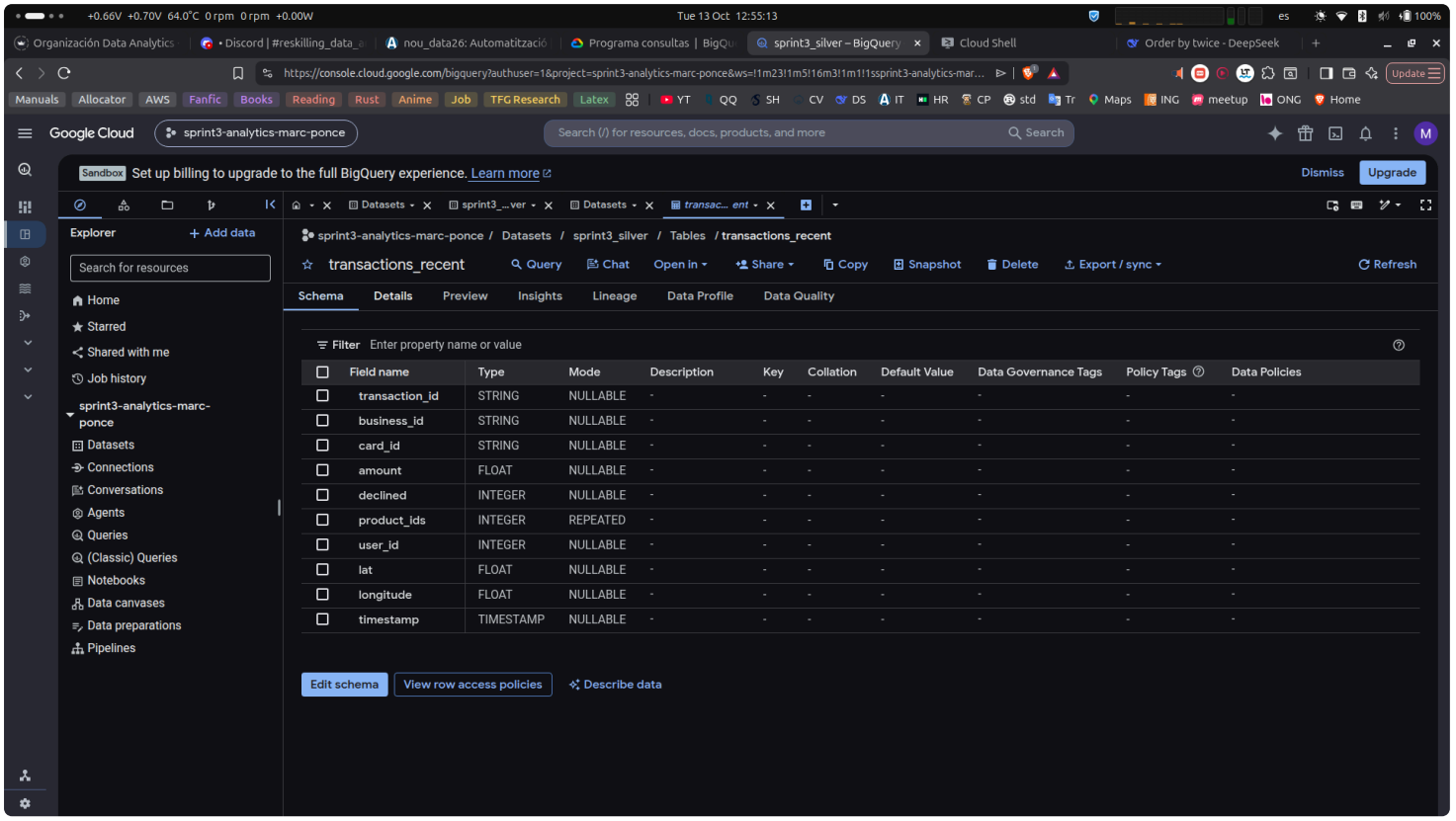
```
CREATE OR REPLACE TABLE
    `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
AS SELECT
    t.* EXCEPT(t.timestamp),
    TIMESTAMP_SUB(
        CURRENT_TIMESTAMP(),
        CAST(RAND() * 50 AS INT64) AS DAY
    ) AS timestamp
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t;
```

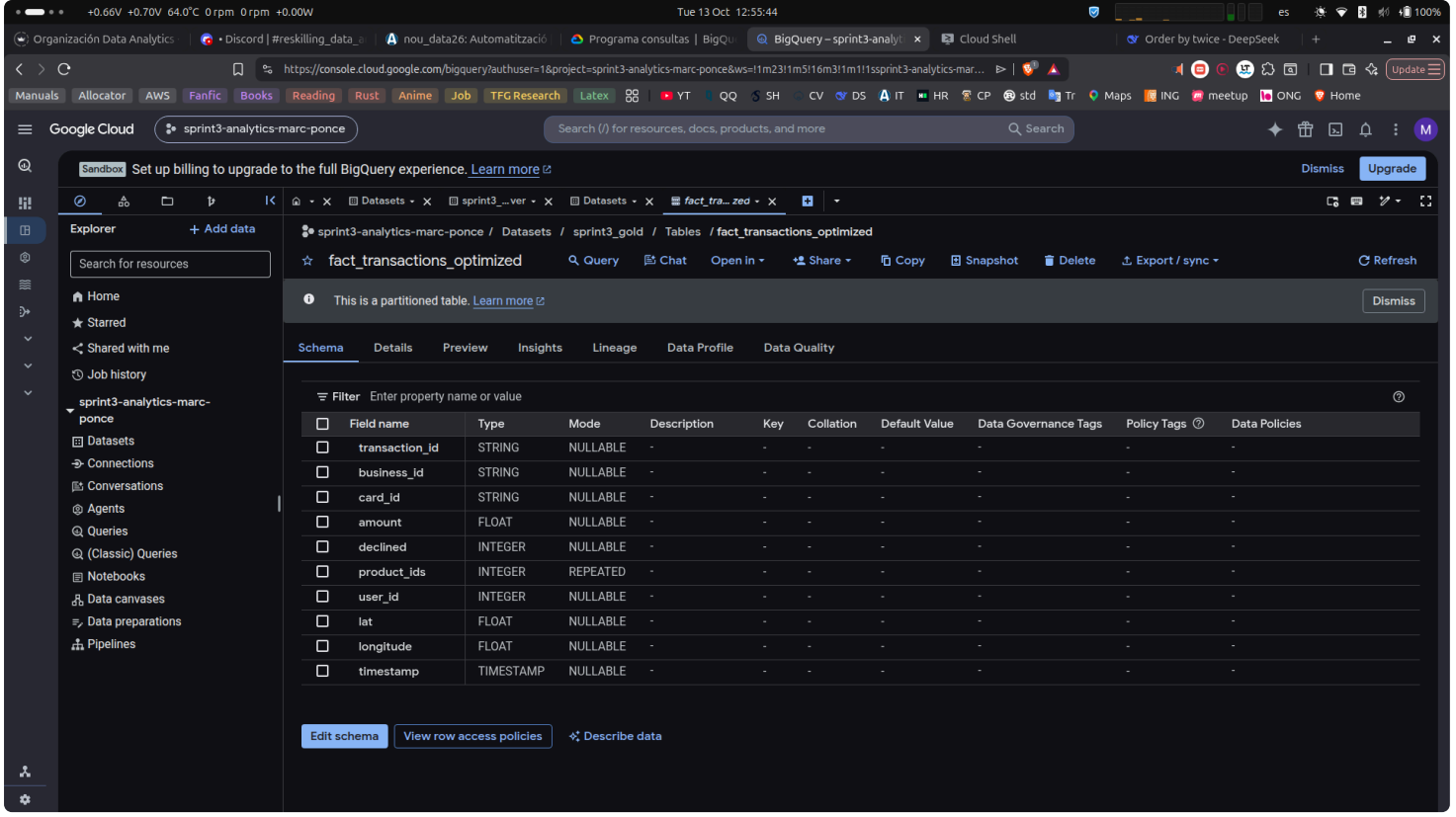
```
CREATE OR REPLACE TABLE
    `sprint3-analytics-marc-ponce.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id
AS (
    SELECT *
    FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
);
```


Cloud Shell Editor
S4L1E2.sql
/* @datacloud.settings
}
*/
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
AS SELECT
t.* EXCEPT(`timestamp`),
TIMESTAMP_SUB(
CURRENT_TIMESTAMP(),
INTERVAL CAST(RAND() * 50 AS INT64) DAY
) AS timestamp
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t;
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id
AS (
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
);
marcponcesochnov@cloudshell:~ (sprint3-analytics-marc-ponce)$ bq query --dry_run < S4L1E2.sql
Query successfully validated. Assuming the tables are not modified, running this query will process lower bound of 11627128 bytes of data.
marcponcesochnov@cloudshell:~ (sprint3-analytics-marc-ponce)$


Cloud Shell Editor
S4L1E2.sql
Connection Validation Failed | This script will process 11.09 MB when run.
/* @datacloud.settings
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
AS SELECT
t.* EXCEPT(`timestamp`),
TIMESTAMP_SUB(
CURRENT_TIMESTAMP(),
INTERVAL CAST(RAND() * 50 AS INT64) DAY
) AS timestamp
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t;
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id
AS (
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
);
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
QUERY RESULTS
CREATE OR REPLAC...
CREATE OR REPLAC...
Job Information
Results
Execution details
This statement created a new table named transactions_recent.


Cloud Shell Editor
S4L1E1.sql
S4L1E2.sql
/* @datacloud.settings
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
AS SELECT
t.* EXCEPT(`timestamp`),
TIMESTAMP_SUB(
CURRENT_TIMESTAMP(),
INTERVAL CAST(RAND() * 50 AS INT64) DAY
) AS timestamp
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t;
CREATE OR REPLACE TABLE
`sprint3-analytics-marc-ponce.sprint3_gold.fact_transactions_optimized`
PARTITION BY DATE(timestamp)
CLUSTER BY business_id
AS (
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent`
);
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
QUERY RESULTS
CREATE OR REPLAC...
CREATE OR REPLAC...
Job Information
Results
Execution details
This statement created a new table named fact_transactions_optimized.

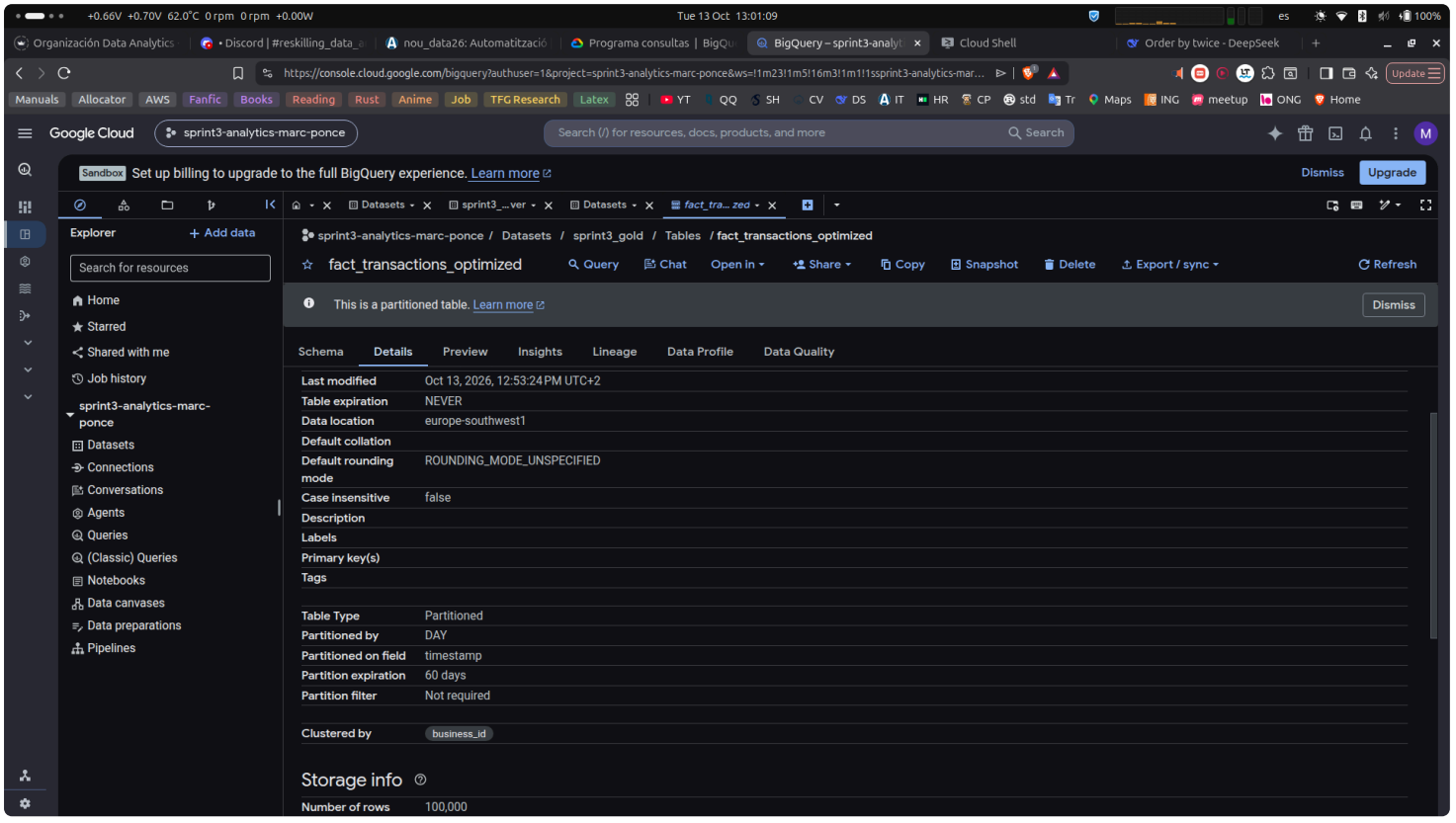

Google Cloud
sprint3-analytics-marc-ponce
Search (/) for resources, docs, products, and more
Sandbox
Set up billing to upgrade to the full BigQuery experience. Learn more
Dismiss
Upgrade
Explorer
+ Add data
Search for resources
Home
Starred
Shared with me
Job history
sprint3-analytics-marc-ponce
Datasets
Connections
Conversations
Agents
Queries
(Classic) Queries
Notebooks
Data canvases
Data preparations
Pipelines
sprint3-analytics-marc-ponce / Datasets / sprint3_silver / Tables / transactions_recent
transactions_recent
Query
Chat
Open in
Share
Copy
Snapshot
Delete
Export / sync
Refresh
Schema
Details
Preview
Insights
Lineage
Data Profile
Data Quality
Filter
Enter property name or value
Field name
Type
Mode
Description
Key
Collation
Default Value
Data Governance Tags
Policy Tags
Data Policies
transaction_id STRING NULLABLE
business_id STRING NULLABLE
card_id STRING NULLABLE
amount FLOAT NULLABLE
declined INTEGER NULLABLE
product_ids INTEGER REPEATED
user_id INTEGER NULLABLE
lat FLOAT NULLABLE
longitude FLOAT NULLABLE
timestamp TIMESTAMP NULLABLE
Edit schema
View row access policies
Describe data



» Details screenshot

"Partitioned by: `DAY` ... on field `timestamp`"

"Clustered by: `business_id`"



Exercici 3: La Prova del Cotó (Benchmark)

> ## Escenari:
>
> Escenari: Ja has creat la taula `sprint3_gold.fact_transactions_optimized` amb les estratègies de Particionament i Clustering.

> Ara toca demostrar a l'equip la millora de rendiment i l'estalvi de costos que has aconseguit. Simula rem una petició típica de negoci: "Volem veure totes les transaccions dels últims 30 dies".

🧩 Tasca:

L'objectiu és clar i directe: **aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional**.

Construeix una consulta SQL que seleccioni totes les columnes (`SELECT *`) i filtri les dades dels últims 30 dies.

⚠️ **NO executis les consultes encara.** En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els **Bytes processats** de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

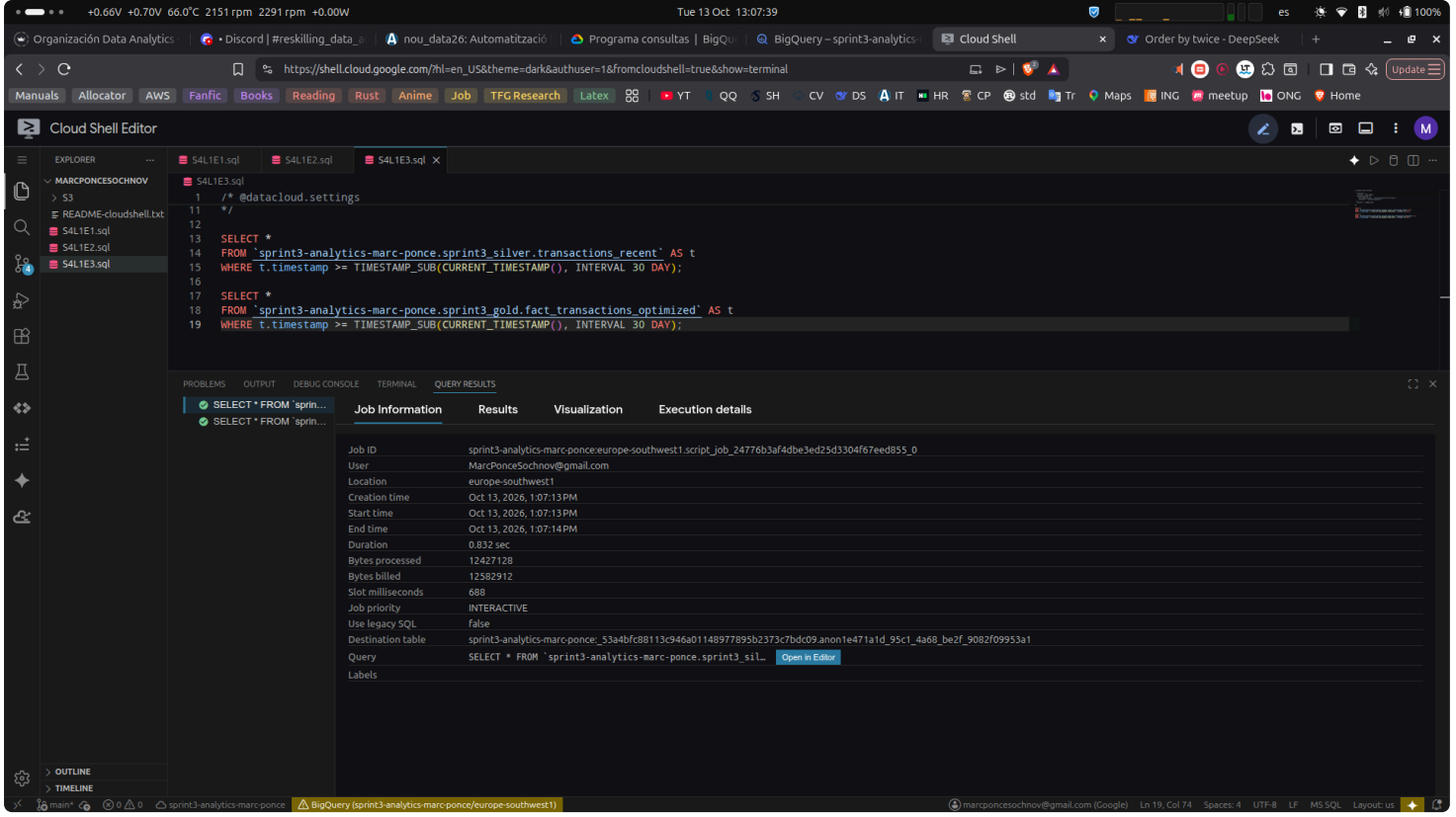
- **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.
- **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el `FROM` perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

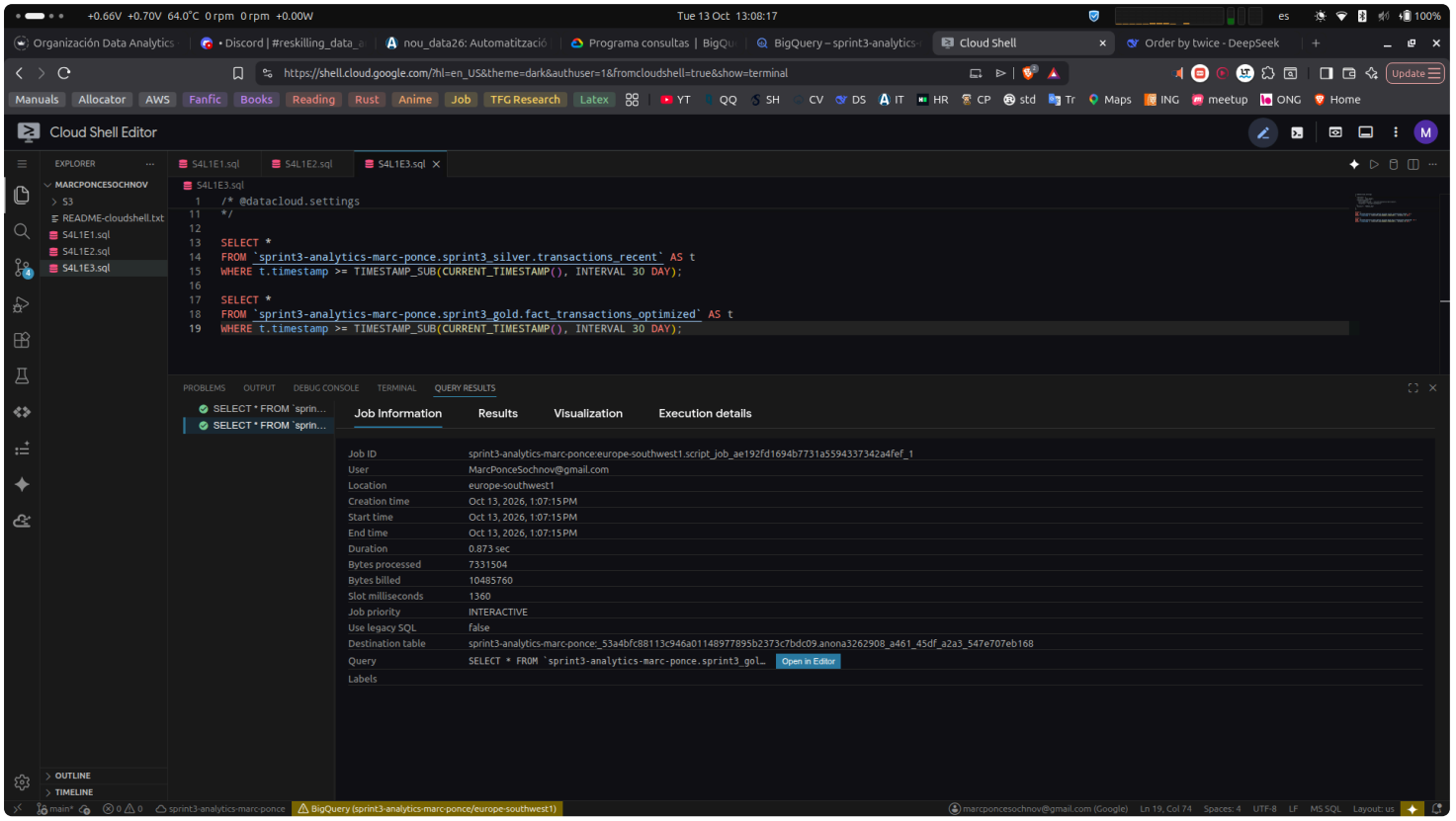
👉 Entrega:

- El codi SQL base que has dissenyat.
  - Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- Un breu text calculant quin és el **% d'estalvi** que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

```sql
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_recent` AS t
WHERE t.timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```

```sql
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_gold.fact_transactions_optimized` AS t
WHERE t.timestamp >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY);
```





» The non-partitioned table's query processed `12427128 bytes`, while the partitioned table's query processed just `7331504 bytes`, just `59%` of the non-partitioned table's query, saving `41%` of compute. Given the `10MB` base cost, the savings amount to just `16.67%` (i.e. `2097152 bytes` or `~2MB`)

Exercici 4: Smart Caching (Vistes Materialitzades)

> ### Escenari:
>
> El Director General té un quadre de comandament que mostra les "**Vendes Totals per Dia**". Ell (i 50 managers més) refresquen aquest gràfic constantment.

Problema:

Si utilitzes una consulta normal o una Vista estàndard, cada vegada que algú refresca la pàgina, BigQuery recalcula la suma de milions de files. És lent i estàs pagant pel mateix càlcul una vegada i una altra.

Solució:

Utilitzar una **Vista Materialitzada**. Aquesta tecnologia guarda el *resultat* del càlcul a la memòria cau i només l'actualitza si entren dades noves (incremental).

🧩 Tasca:

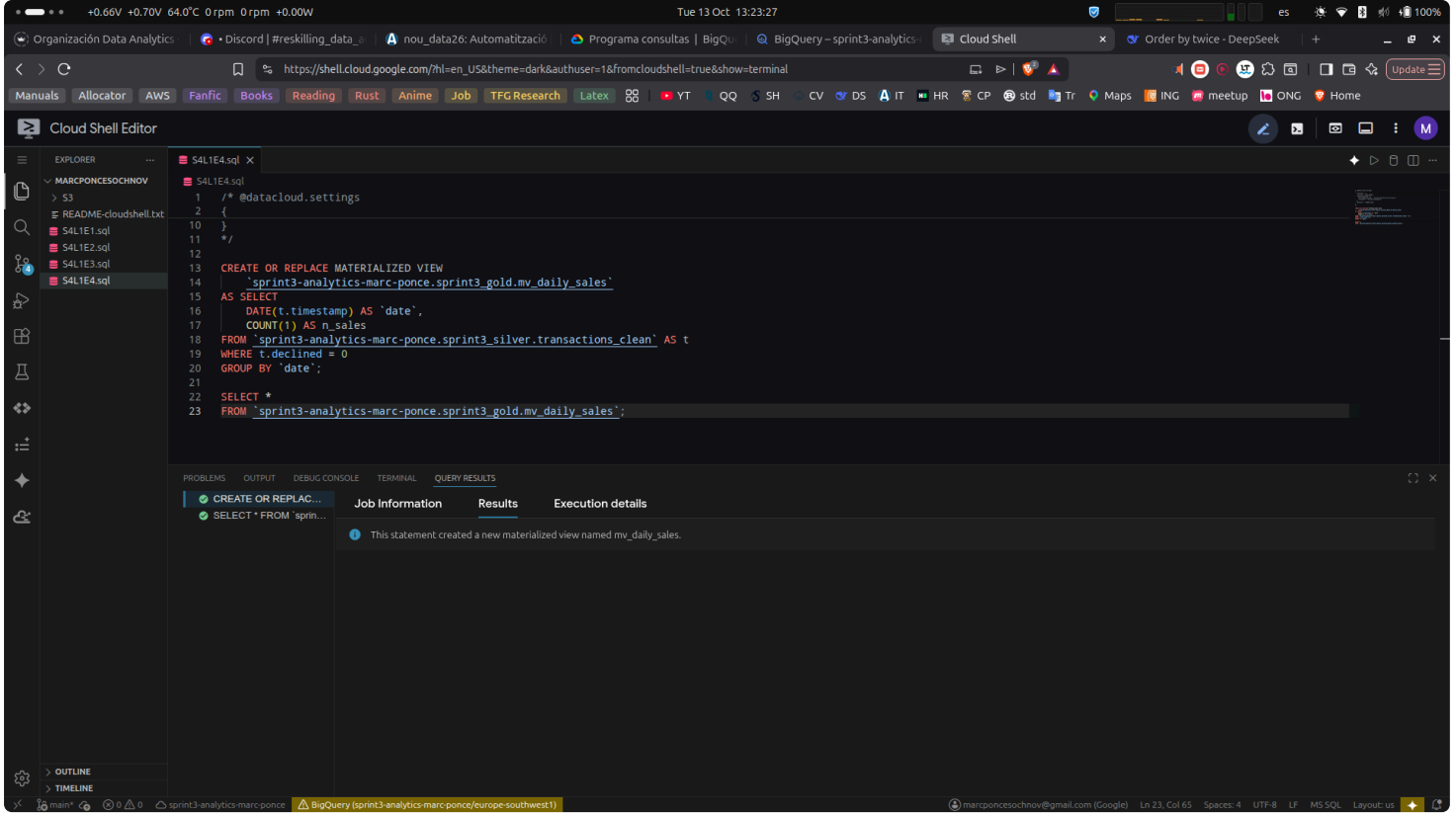
- Crea una **Vista Materialitzada** anomenada `sprint3_gold.mv_daily_sales` que mostri les **Vendes Totals per Dia**.

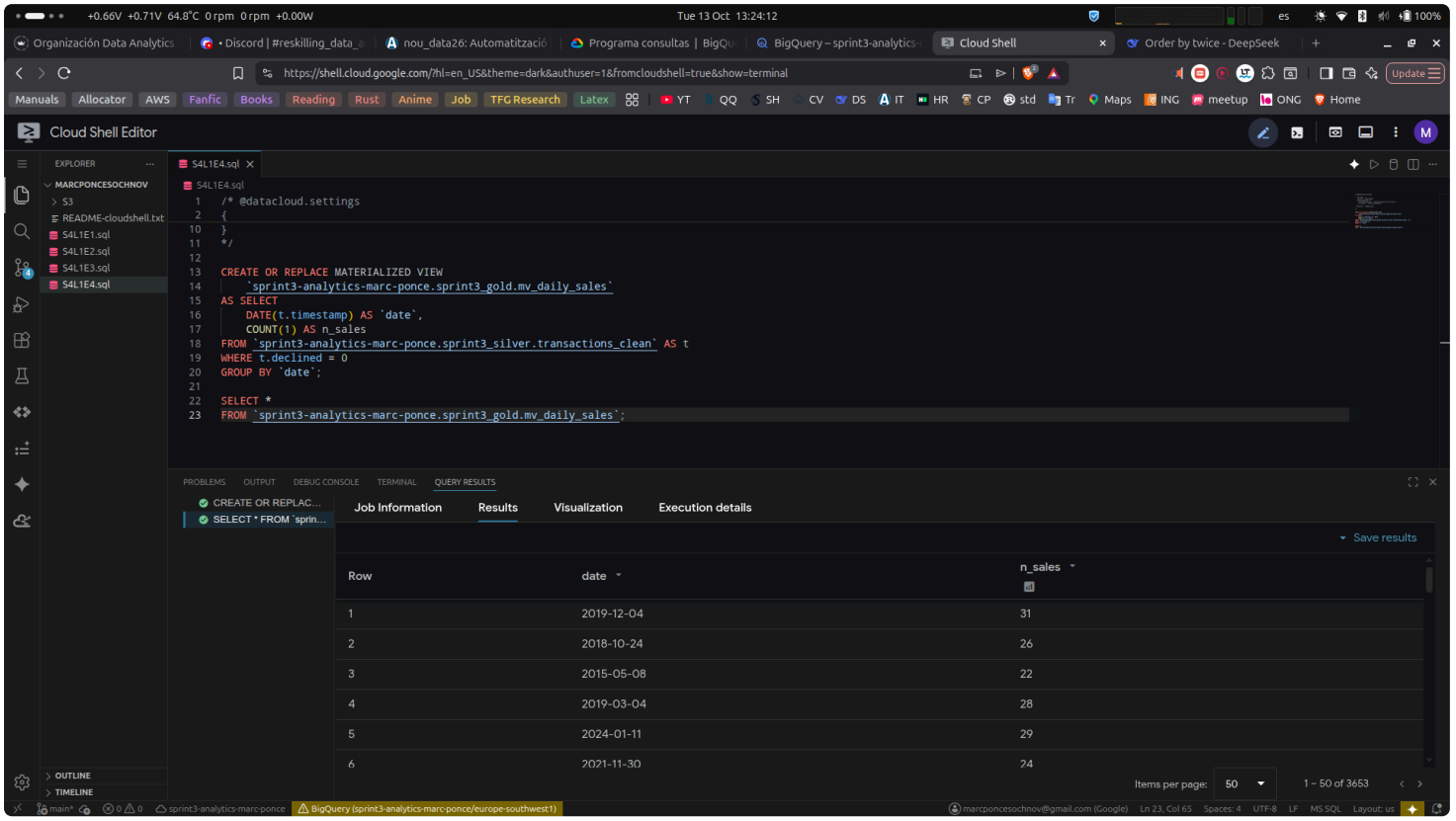
👉 Entrega:

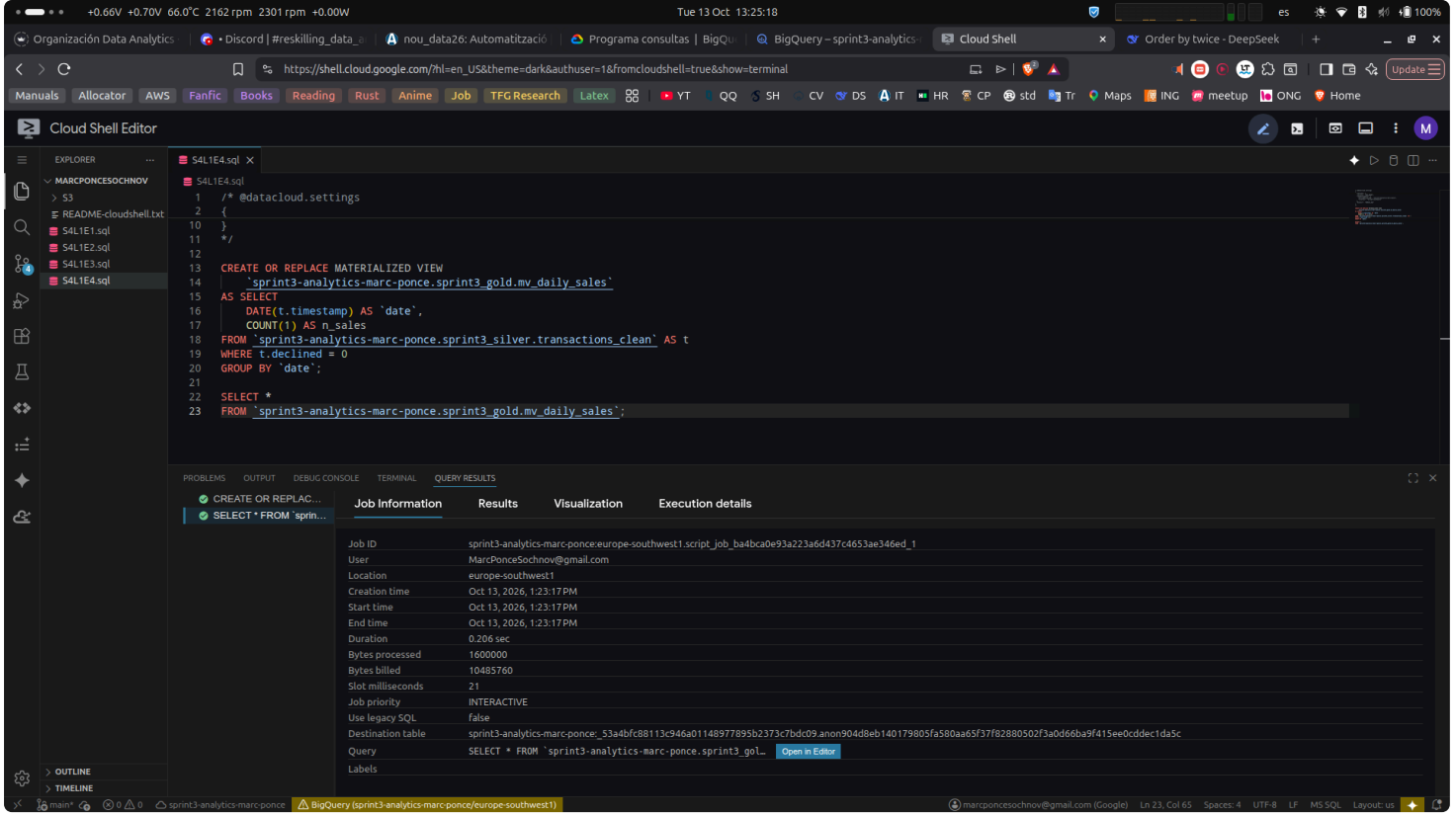
- Codi per Crear la **Vista Materialitzada**.
- Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

```
CREATE OR REPLACE MATERIALIZED VIEW
    `sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales`
AS SELECT
    DATE(t.timestamp) AS `date`,
    COUNT(1) AS n_sales
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
WHERE t.declined = 0
GROUP BY `date`;
```

```
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales`;
```


Cloud Shell Editor
S4L1E4.sql
/* @datacloud.settings
{
}
*/
CREATE OR REPLACE MATERIALIZED VIEW
`sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales`
AS SELECT
DATE(t.timestamp) AS `date`,
COUNT(1) AS n_sales
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
WHERE t.declined = 0
GROUP BY `date`;
SELECT *
FROM `sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales`;
QUERY RESULTS
Job Information
Results
Execution details
This statement created a new materialized view named mv_daily_sales.


Cloud Shell Editor
QUERY RESULTS
Job Information
Results
Visualization
Execution details
Save results
Row
date
n_sales
1 2019-12-04 31
2 2018-10-24 26
3 2015-05-08 22
4 2019-03-04 28
5 2024-01-11 29
6 2021-11-30 24
Items per page: 50
1 – 50 of 3653



Nivell 2: SQL Analític Avançat

> ## Escenari:
>
> La direcció comença a fer preguntes que no es resolen amb un simple `GROUP BY`. Necessitem anàlisi de tendències i comportament d'usuari.

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

> ## Escenari:
>
> L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a €500. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

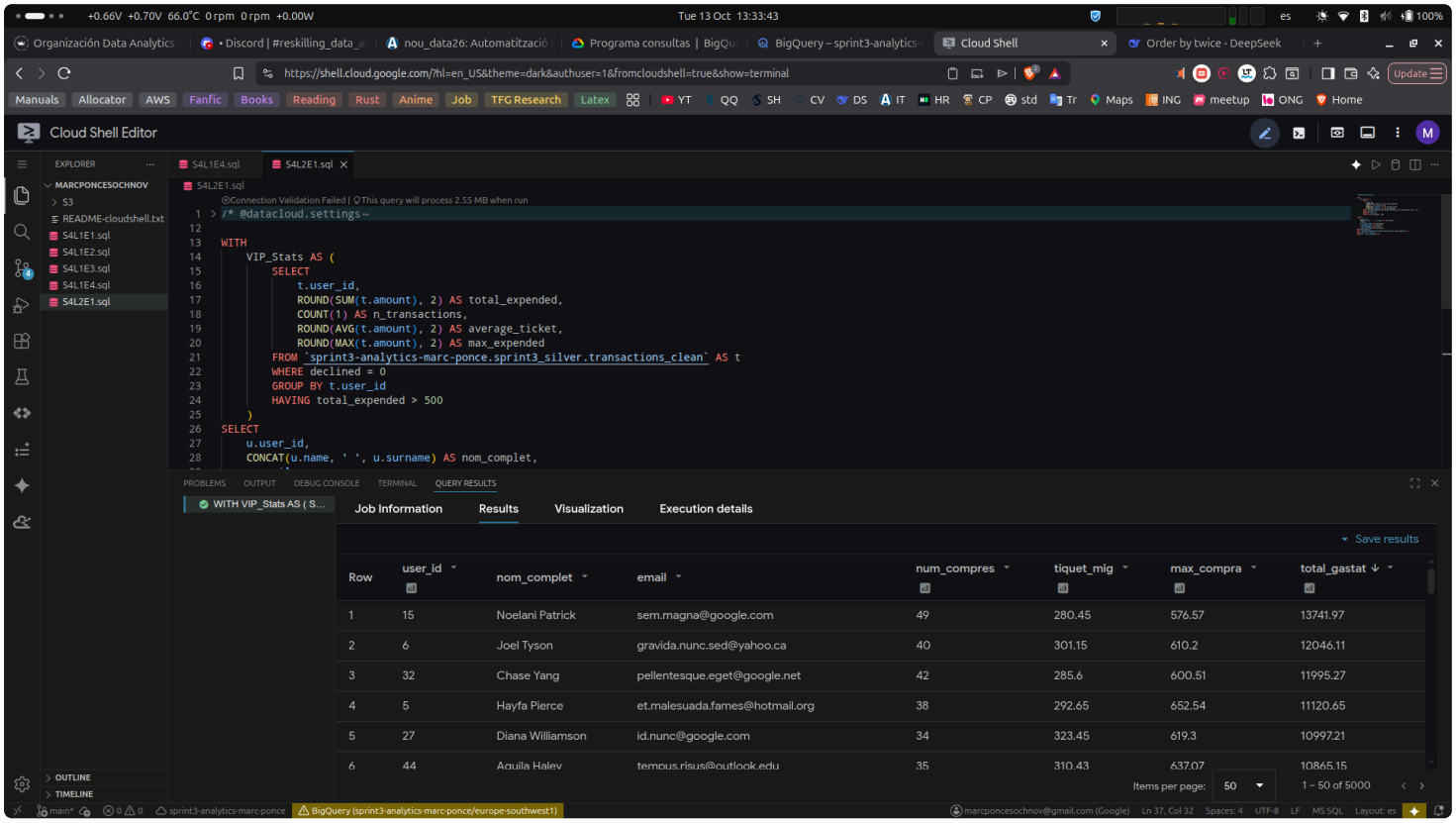
🧩 Tasca:

- **1. Crea una CTE anomenada `VIP_Stats` que agrupi per usuari i calculi**:
- La Despesa Total (`SUM`).
- La Quantitat de Transaccions (COUNT).
- El Tiquet Mitjà (`AVG`), arrodonit a 2 decimals.
- La Compra Màxima (`MAX`).
- Filtre: Mantén només aquells la Despesa Total dels quals sigui `> 500`.
- **Creua la CTE amb `users_combined` per obtenir les dades personals.**

Requisits de Sortida:

- Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenat per total_gastat descendent.

```sql
WITH
    VIP_Stats AS (
        SELECT
            t.user_id,
            ROUND(SUM(t.amount), 2) AS total_expended,
            COUNT(1) AS n_transactions,
            ROUND(AVG(t.amount), 2) AS average_ticket,
            ROUND(MAX(t.amount), 2) AS max_expended
        FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
        WHERE declined = 0
        GROUP BY t.user_id
        HAVING total_expended > 500
    )
SELECT
    u.user_id,
    CONCAT(u.name, ' ', u.surname) AS nom_complet,
    u.email,
    v.n_transactions AS num_compres,
    v.average_ticket AS tiquet_mig,
    v.max_expended AS max_compra,
    v.total_expended AS total_gastat
FROM VIP_Stats AS v
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.users_combined` AS u
ON v.user_id = u.user_id
ORDER BY v.total_expended DESC;
```



Row	user_id	nom_complet	email	num_compres	tiquet_mig	max_compra	total_gastat
1	15	Noelani Patrick	sem.magna@google.com	49	280.45	576.57	13741.97
2	6	Joel Tyson	gravida.nunc.sed@yahoo.ca	40	301.15	610.2	12046.11
3	32	Chase Yang	pellentesque.eget@google.net	42	285.6	600.51	11995.27
4	5	Hayfa Pierce	et.malesuada.fames@hotmail.org	38	292.65	652.54	11120.65
5	27	Diana Williamson	id.nunc@google.com	34	323.45	619.3	10997.21
6	44	Aquila Haley	tempus.risus@outlook.edu	35	310.43	637.07	10865.15

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

> **Escenari:**

> La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).

📈 Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalcular sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.

🧩 Tasca:

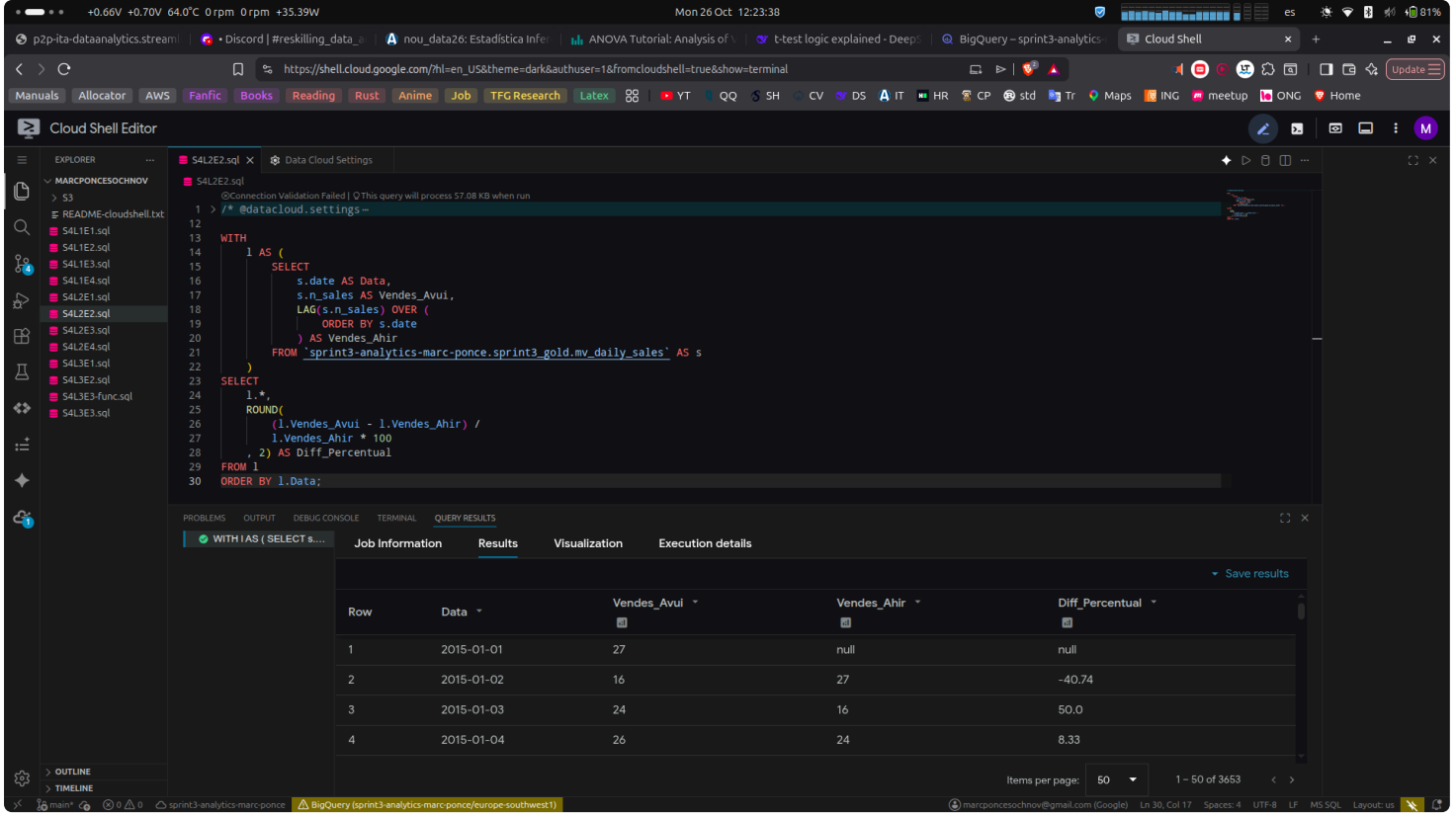
Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

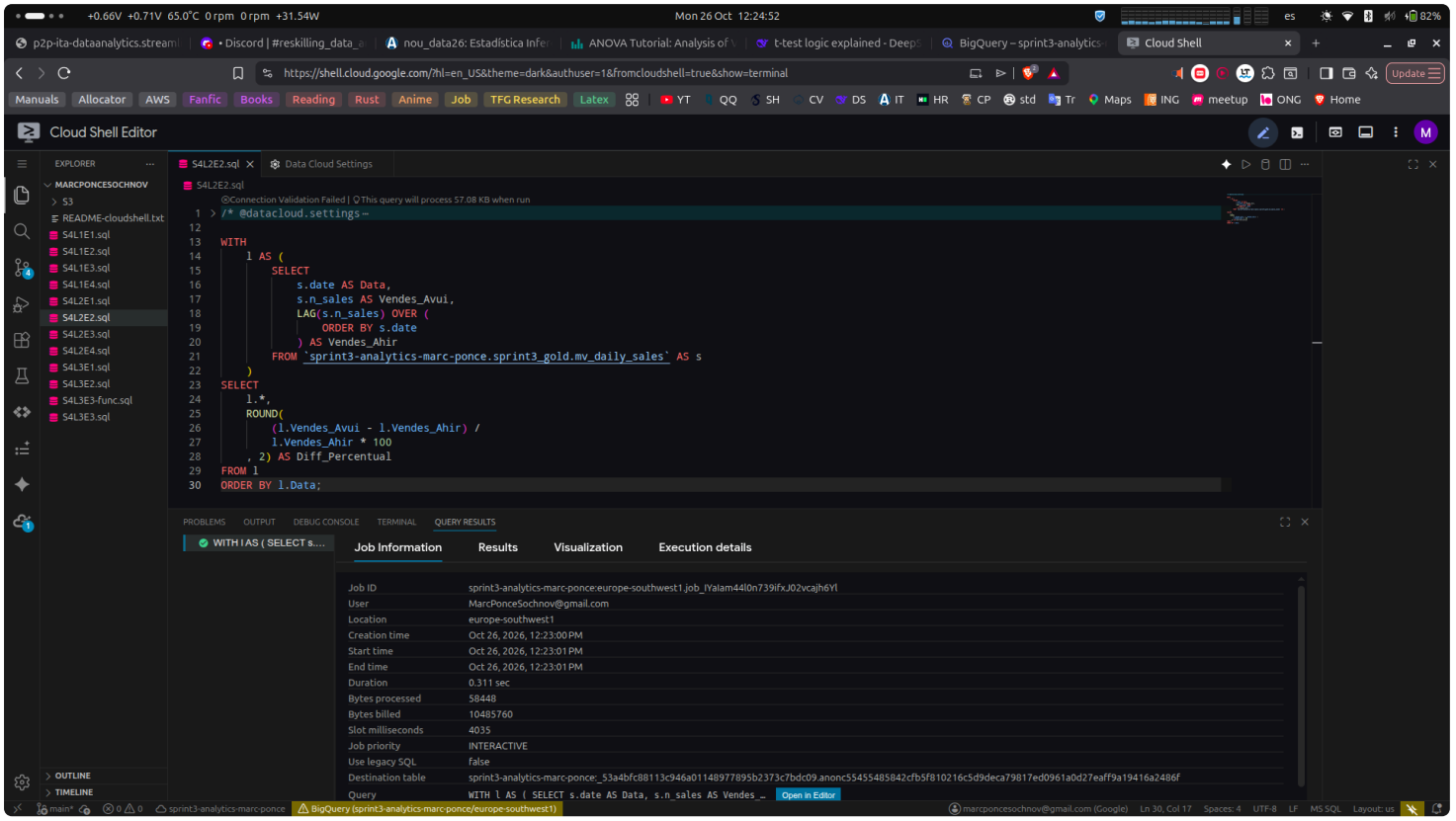
1. `Data`: La data de venda.
2. `Vendes_Avui` El total venut aquell dia.
3. `Vendes_Ahir`: El total venut el dia anterior (usant funcions de finestra).
4. `Diff_Percentual`: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.

💡 Pista Tècnica:

Utilitza la funció `LAG(camp) OVER (ORDER BY data)` per llegir el valor de la fila anterior sense necessitat de fer `JOINs` complexos.

```
WITH
    l AS (
        SELECT
            s.date AS Data,
            s.n_sales AS Vendes_Avui,
            LAG(s.n_sales) OVER (
                ORDER BY s.date
            ) AS Vendes_Ahir
        FROM `sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales` AS s
    )
SELECT
    l.*,
    ROUND(
        (l.Vendes_Avui - l.Vendes_Ahir) /
        l.Vendes_Ahir * 100
    , 2) AS Diff_Percentual
FROM l
ORDER BY l.Data;
```





Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

> ## Escenari:
>
> El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

🧩 Tasca:

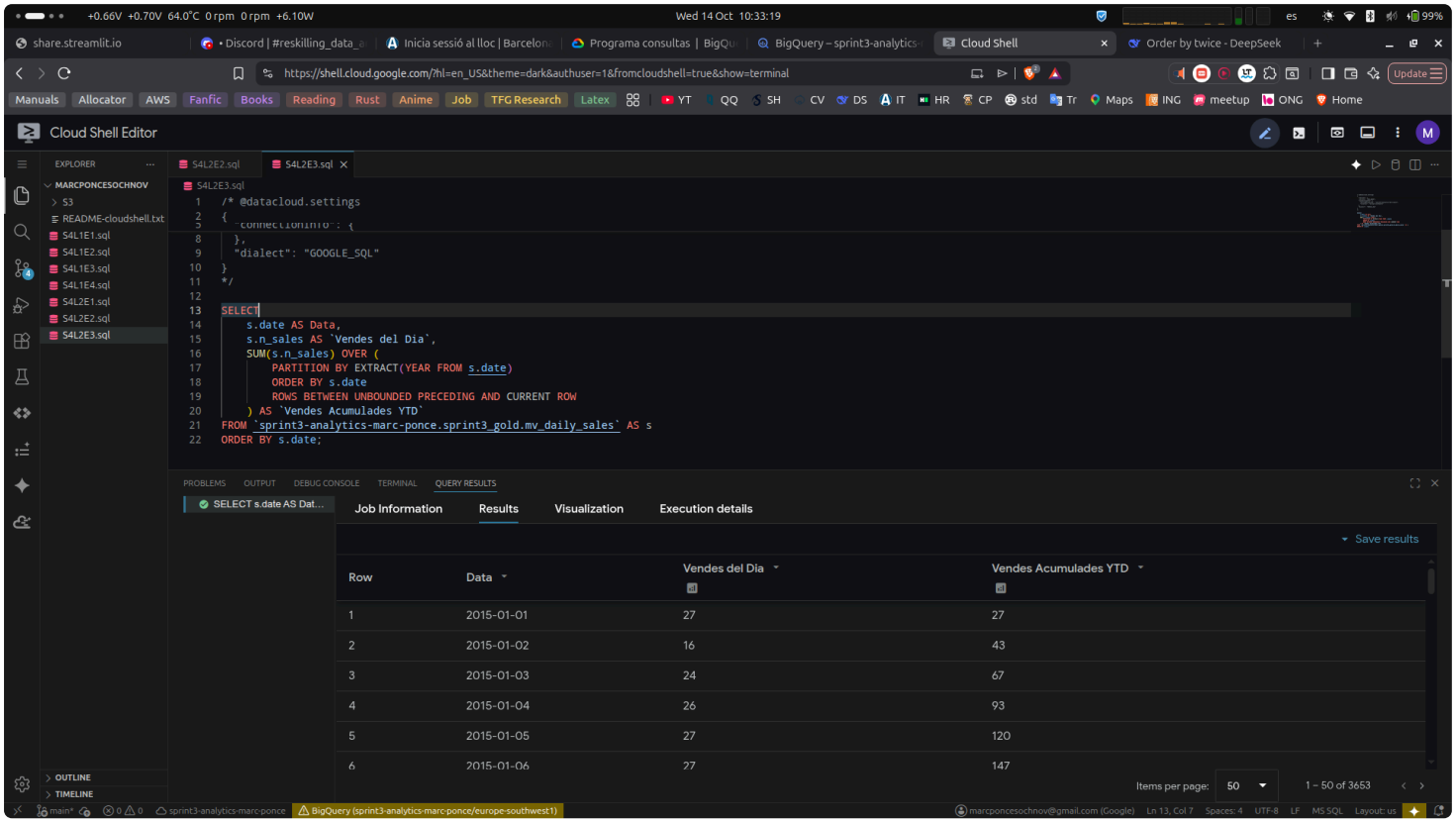
Utilitzant la vista materialitzada `mv_daily_sales` (per no recalcular agregacions base), genera un informe amb tres columnes:

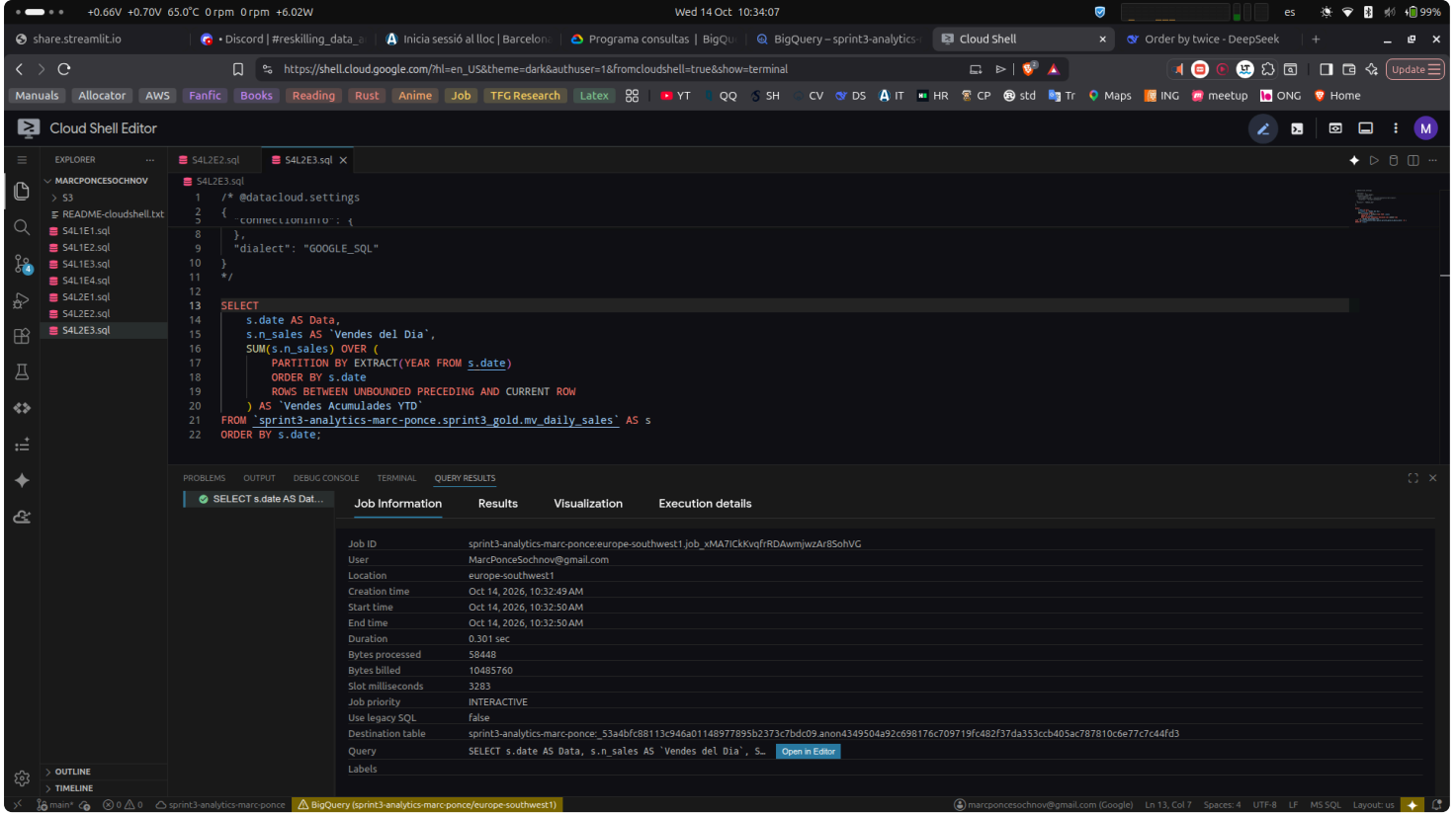
1. `Data`
2. `'Vendes del Dia'` : Arrodonit a 2 decimals.
3. `'Vendes Acumulades YTD'` : Una suma progressiva de les vendes que ha de reiniciar-se cada **1 de Gener**. El resultat final ha d'estar arrodonit a 2 decimals.

💡 Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant `PARTITION BY EXTRACT(YEAR FROM data)`. L'estructura completa seria: `SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`.

```sql
SELECT
    s.date AS Data,
    s.n_sales AS `Vendes del Dia`,
    SUM(s.n_sales) OVER (
        PARTITION BY EXTRACT(YEAR FROM s.date)
        ORDER BY s.date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS `Vendes Acumulades YTD`
FROM `sprint3-analytics-marc-ponce.sprint3_gold.mv_daily_sales` AS s
ORDER BY s.date;
```





Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

> ## Escenari:
>
> L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

🧩 Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

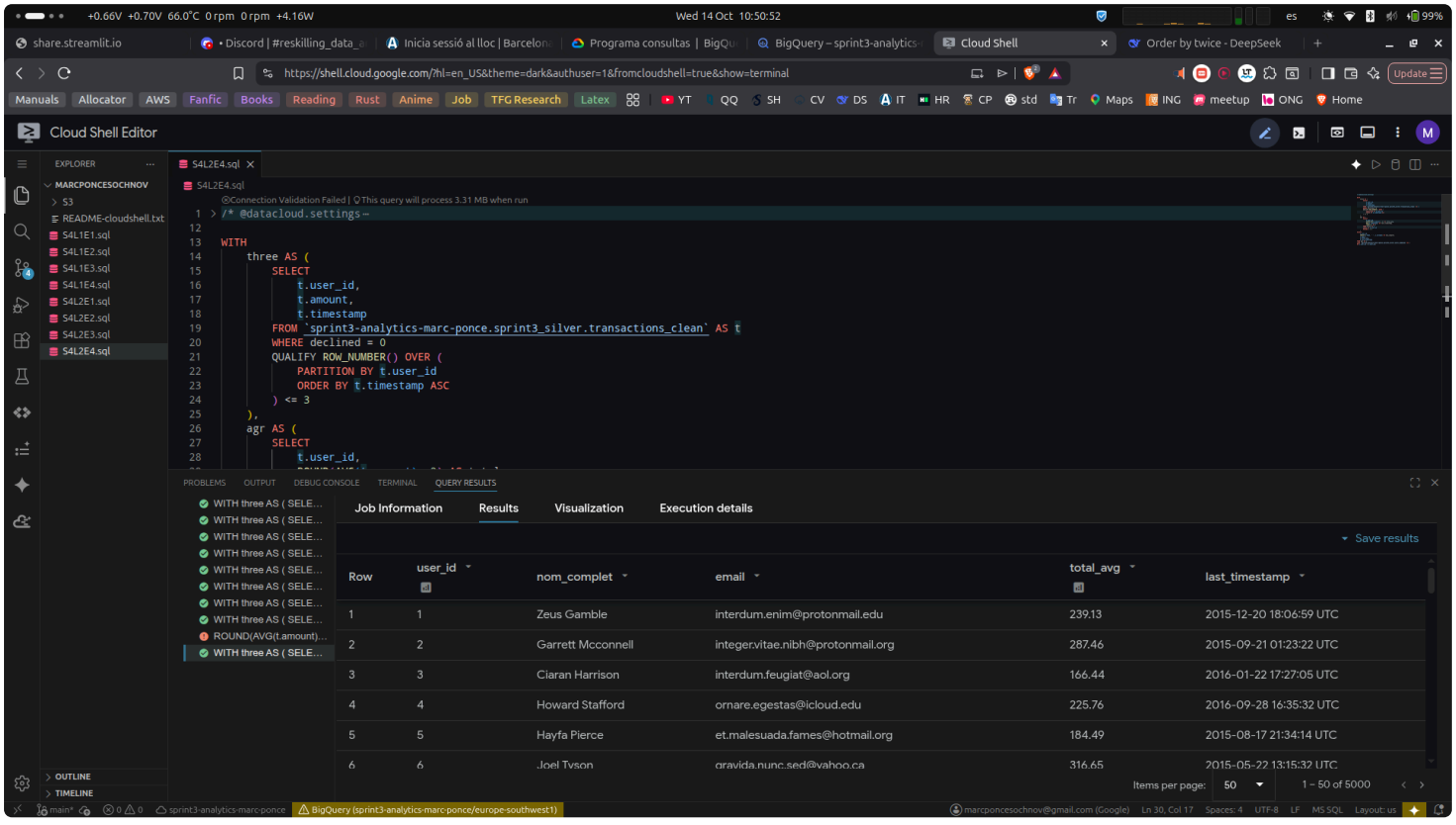
1. **Dades d'usuari (`user_id`, `nom_complet`, `email`)**.
2. **La data i l'import exacte de la 3a compra**.
3. **Mitjana de les 3 primeres**: La mitjana de despesa de les seves transaccions 1, 2 i 3.

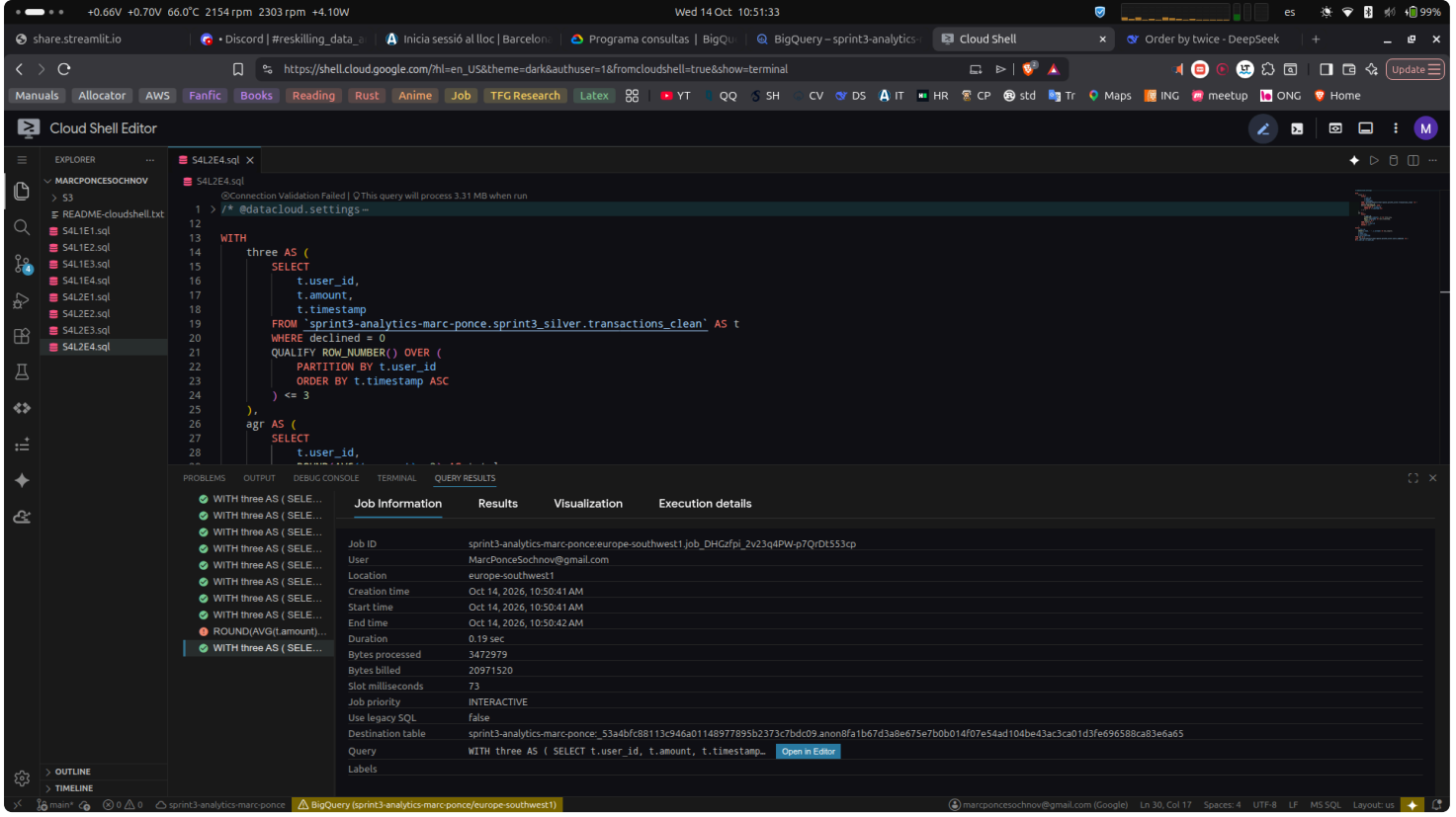
💡 Pista Tècnica:

- **Has d'utilitzar la funció `ROW_NUMBER()` per establir l'ordre cronològic de les compres**.
- **Has d'utilitzar la clàusula `QUALIFY` per filtrar**.
- **Repte d'Optimització**: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

```
WITH
    three AS (
        SELECT
            t.user_id,
            t.amount,
            t.timestamp
```

```sql
        FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
        WHERE declined = 0
        QUALIFY ROW_NUMBER() OVER (
            PARTITION BY t.user_id
            ORDER BY t.timestamp ASC
        ) <= 3
    ),
    agr AS (
        SELECT
            t.user_id,
            ROUND(AVG(t.amount), 2) AS total_avg,
            MAX(t.timestamp) AS last_timestamp,
            COUNT(1) AS n
        FROM three AS t
        GROUP BY t.user_id
        HAVING n = 3
    )
SELECT
    u.user_id,
    CONCAT(u.name, ' ', u.surname) AS nom_complet,
    u.email,
    a.total_avg,
    a.last_timestamp
FROM agr AS a
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.users_combined` AS u
ON u.user_id = a.user_id;
```





» Since our calculation is only interested in the first three transactions of a user, first thing should be to drop those transactions that are of no interest. Afterward we aggregate the filtered result: given it is possible that some users only have one or two transactions, we filter those users out, leaving in our calculation only those who have 3 or more transactions. Optimally I would have dropped them before the aggregation, but I judged that the extra compute of a SUM and a MAX is better done in one run, or wasted, than requiring a second linear read, data copying, etc. Lastly we just perform the join, which order shouldn't matter since both tables are of roughly the same size (I am baffled that this could affect performance, given that I would expect an abstracted language like `SQL` to do those optimizations in the background. It would take an `if else` given the metadata of the tables' sizes to swap them one or another way to extract the best performance.) I've further investigated why order would matter, and it seems that given that the optimizations are based on statistics and metadata that might be stale, a workaround is to always leave small tables last in a join so that the broadcast join is more likely to trigger. Usually if the metadata is not stale for some reason, the optimizer should be able to handle it.

Nivell 3: Analytics Engineering (Arrays & Automatització)

> ### Escenari:
>
> Màrqueting necessita urgentment un Rànquing de Productes més Venuts.

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

> ### Escenari:
>
> Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

🧩 Tasca:

Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

⚙️ Passos Tècnics:

1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
2. Realitza un `JOIN` amb la taula products per obtenir el nom (name) i preu (price) de cada ítem.

   📍 *Nota: Ves amb compte amb els tipus de dades en fer el `JOIN (Array de INT64 vs Product_ID STRING)`.*
3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

✅ Resultat Esperat (Visualització):

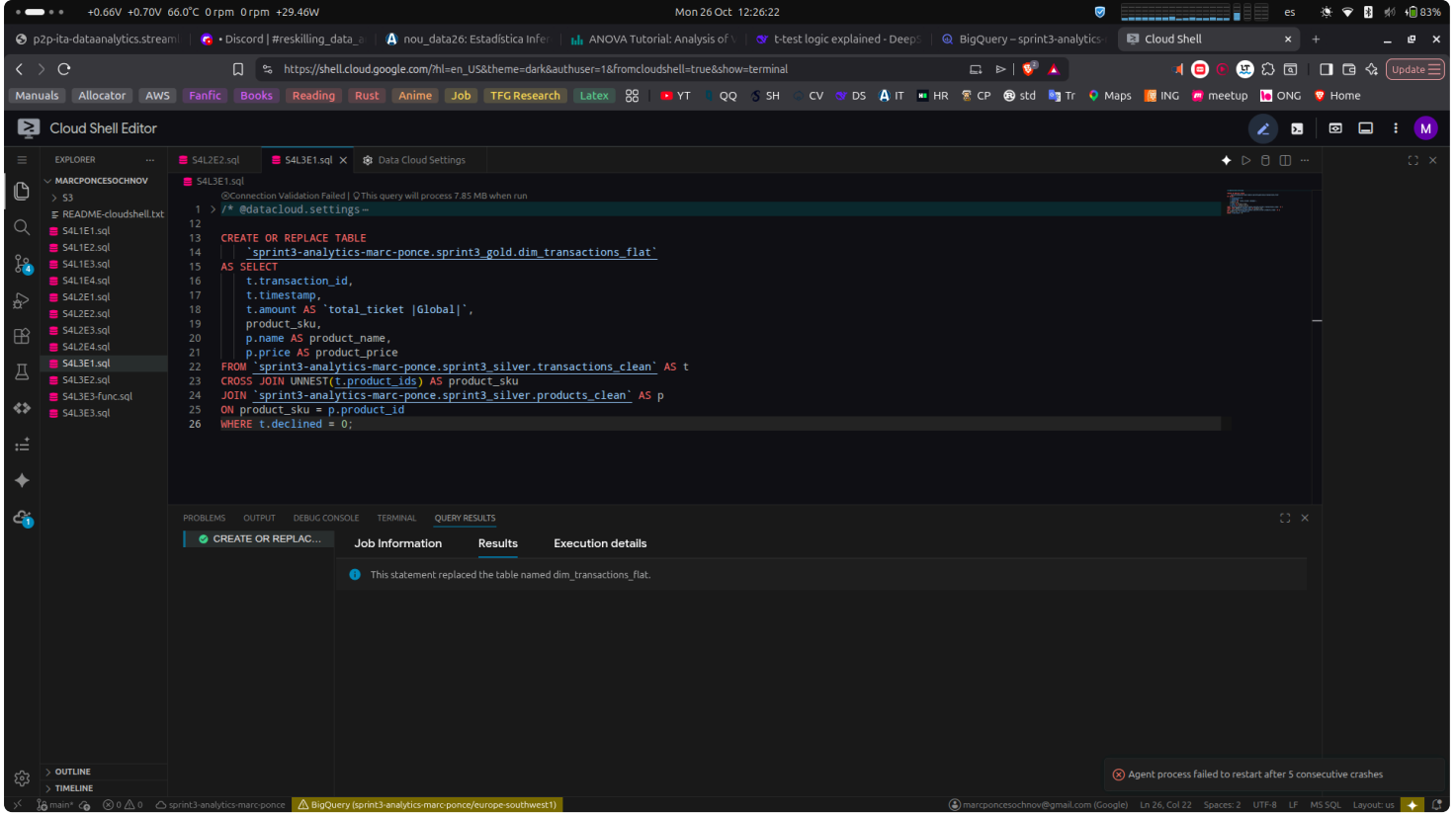
La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

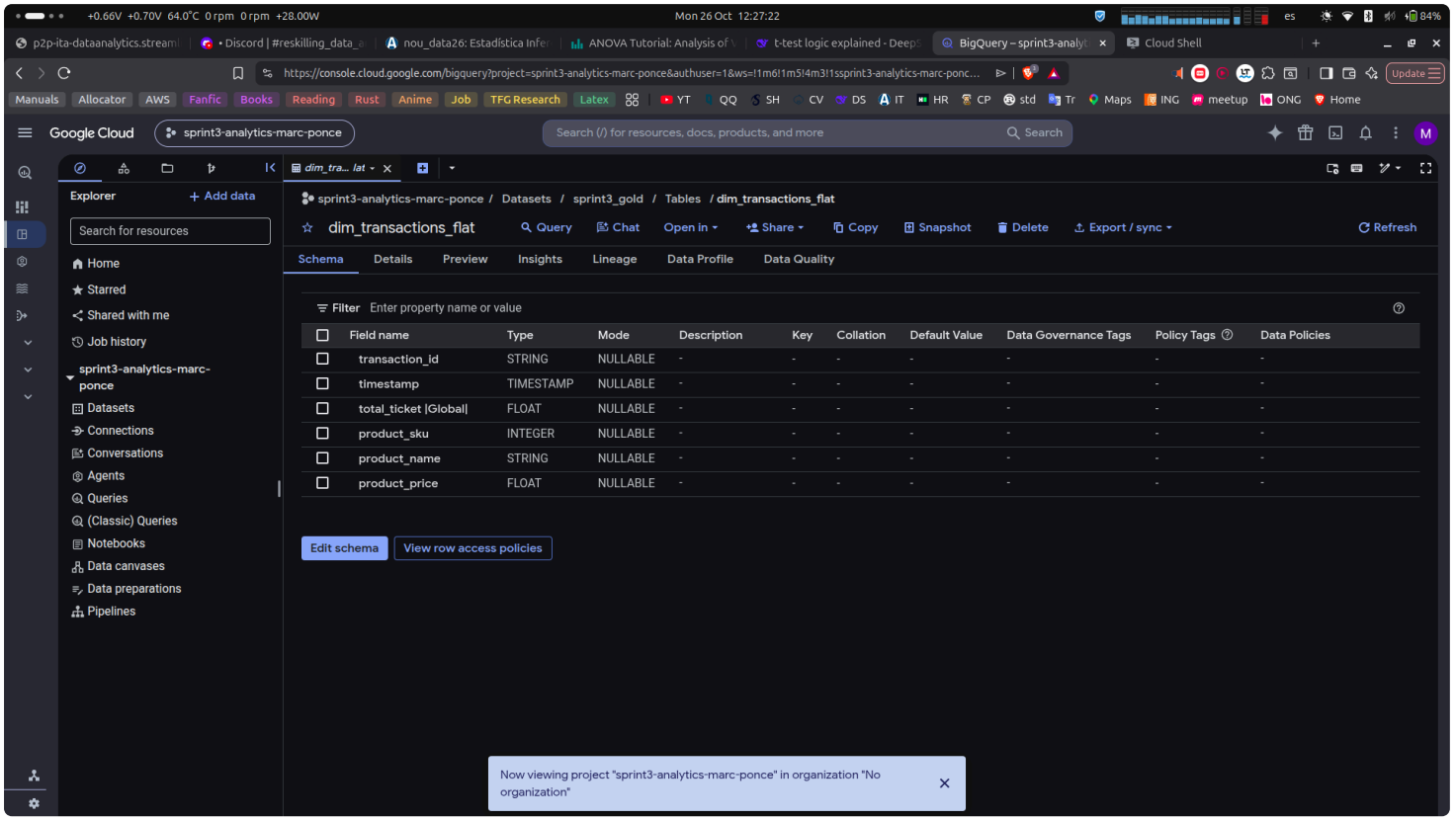
transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

👉 Entrega:

- Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- Una captura de pantalla on es vegi l'esquema de la nova taula.

```
CREATE OR REPLACE TABLE
    `sprint3-analytics-marc-ponce.sprint3_gold.dim_transactions_flat`
AS SELECT
    t.transaction_id,
    t.timestamp,
    t.amount AS `total_ticket |Global|`,
    product_sku,
    p.name AS product_name,
    p.price AS product_price
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
CROSS JOIN UNNEST(t.product_ids) AS product_sku
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.products_clean` AS p
ON product_sku = p.product_id
WHERE t.declined = 0;
```





» `total_ticket (Global)` is an illegal field name because parenthesis are explicitly forbidden for some reason. Given that there doesn't seem a fast way around it, I think management should settle with `total_ticket |Global|`, unless they think that the parenthesis are worth the time.

Exercici 2: El Rànquing de Vendes (Agregació Simple)

> ### Escenari:
>
> Ara que has preparat la infraestructura i disposes de la taula dim_transactions_flat (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

🧩 Tasca:

Genera el **Top 5 de productes** més venuts en la història de la companyia.

🛠️ Mètode:

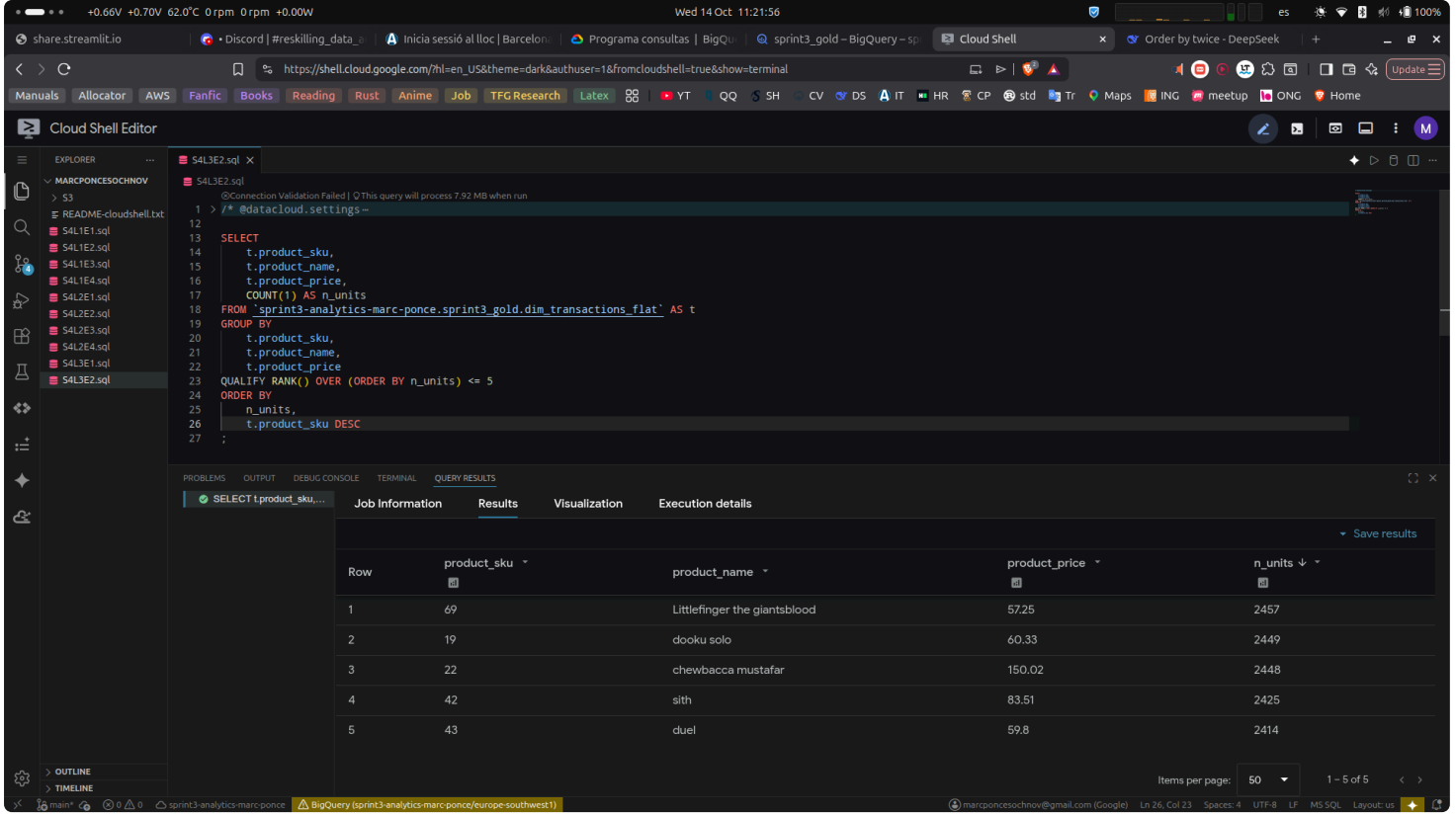
Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

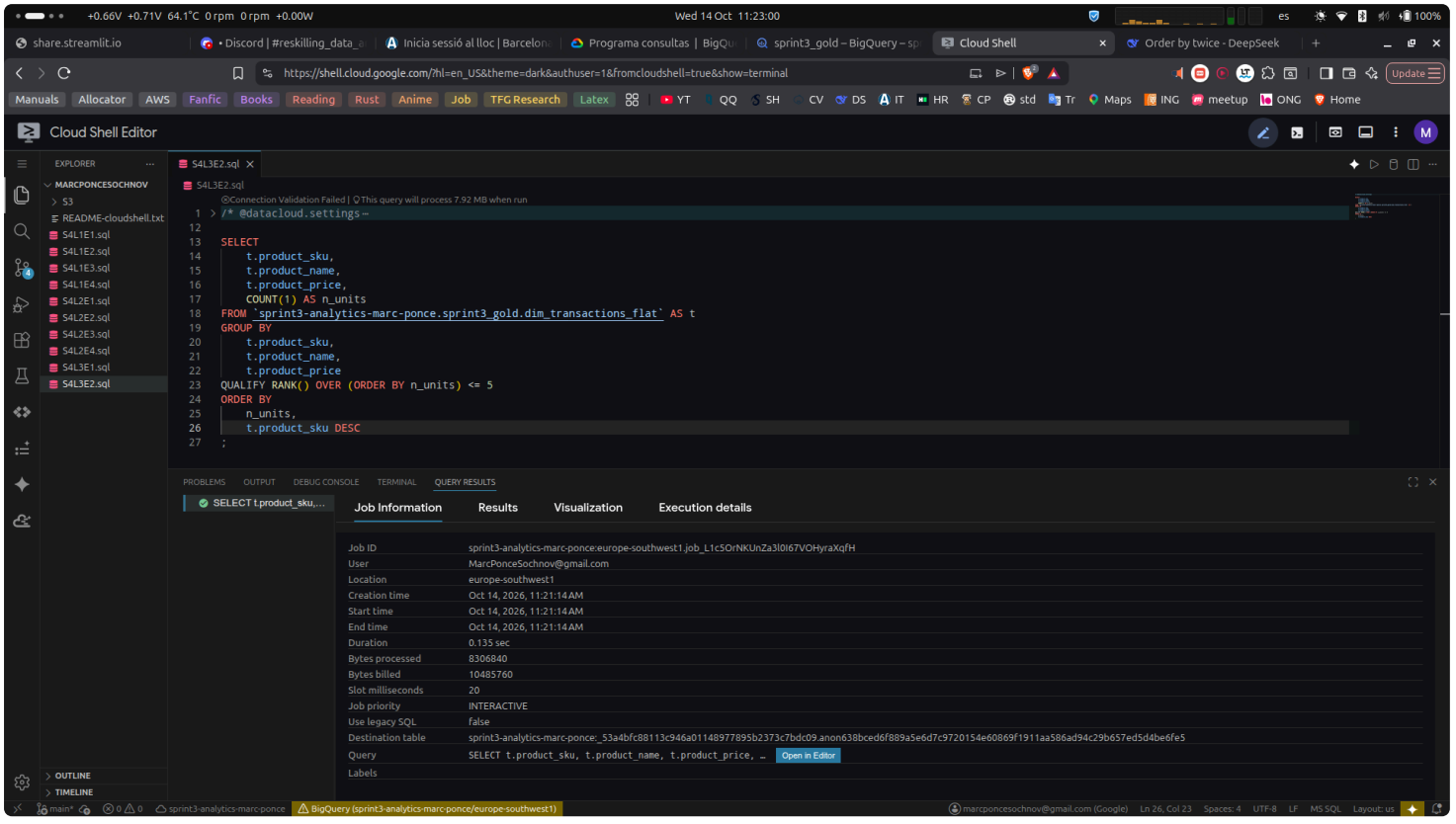
- En ser la taula plana, simplement necessites una agrupació estàndard (`GROUP BY`) pel nom del producte i un compte (`COUNT`) de les files.

👉 Entrega:

- El codi SQL que genera el rànquing.
- Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

```sql
SELECT
    t.product_sku,
    t.product_name,
    t.product_price,
    COUNT(1) AS n_units
FROM `sprint3-analytics-marc-ponce.sprint3_silver.dim_transactions_flat` AS t
GROUP BY
    t.product_sku,
    t.product_name,
    t.product_price
QUALIFY RANK() OVER (ORDER BY n_units) <= 5
ORDER BY
    n_units,
    t.product_sku DESC
;
```





» Given that this is a ranking, I decided to use the `RANK()` function to include more than five products in the possibility that there's a tie.

Exercici 3: Automatització del Pipeline i Visualització

> ### Escenari:
>
> L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:
>
> **1. Lògica Centralitzada**: Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul preu * 1,21 directament en el codi

("Hardcoded"). Has d'utilitzar una funció reutilitzable.
2. Frescor de Dades: El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

🧩 Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

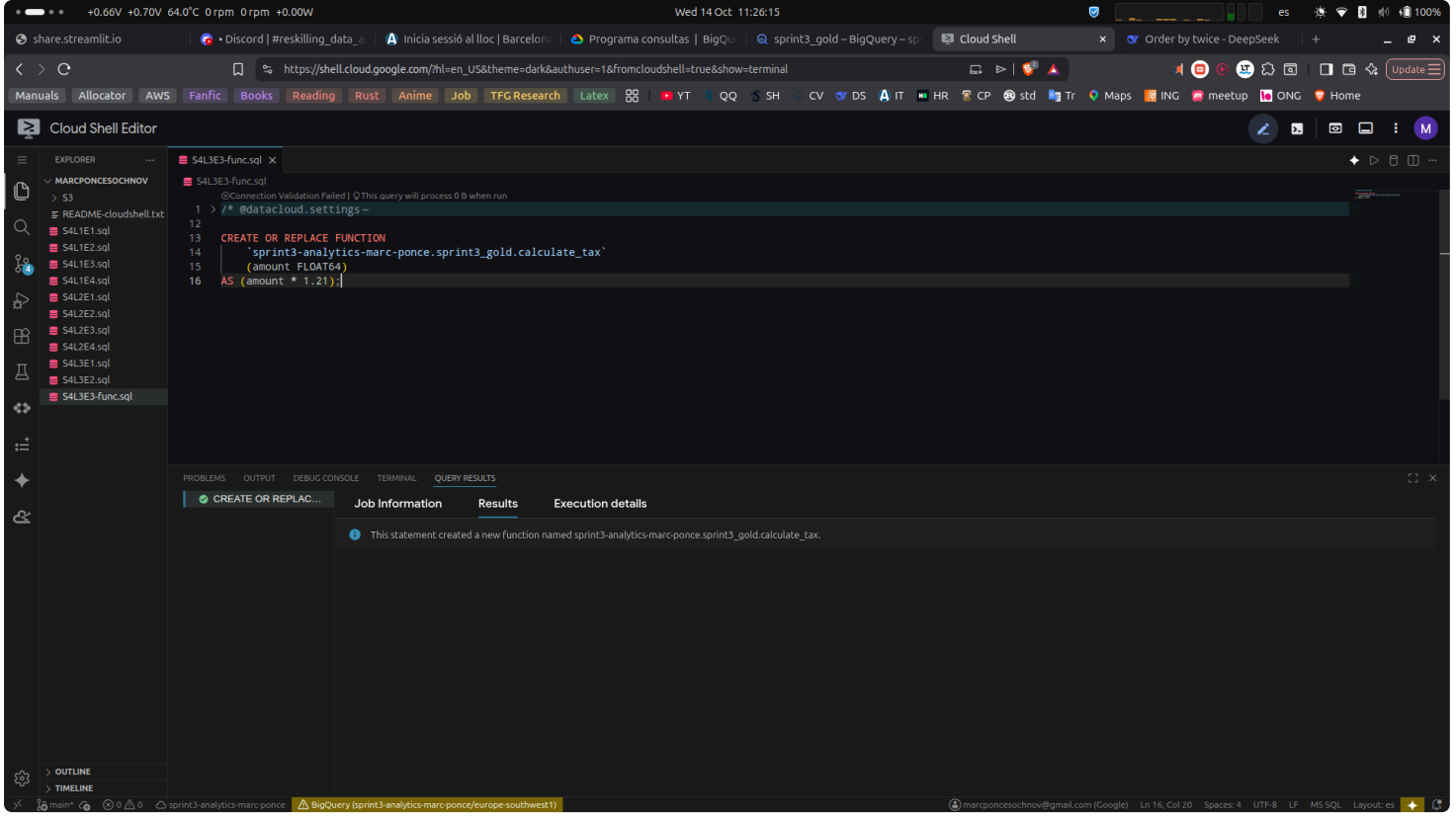
⚙️ Passos Tècnics:

1. **User Defined Functions (UDF)**: Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
2. **Integració i Orquestració**:
   - Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
   - Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
     3. **Visualització (BI)**: Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

👉 Entrega:

- El codi SQL de la UDF.
- El codi SQL actualitzat de la creació de la taula.
- Una captura de pantalla demostrant que la Scheduled Query està programada.
- Un enllaç del gràfic a Looker Studio.

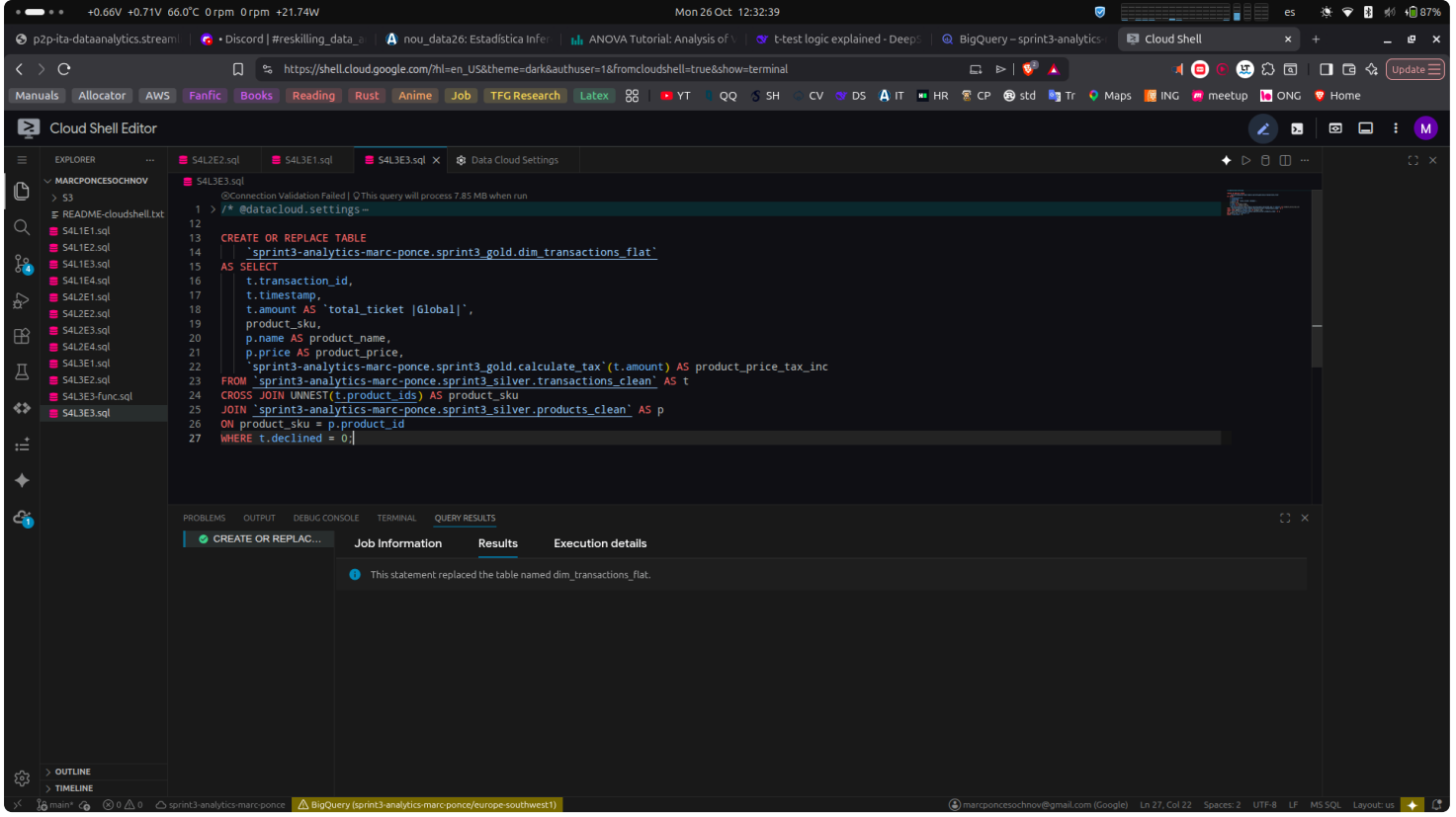
```sql
CREATE OR REPLACE FUNCTION
    `sprint3-analytics-marc-ponce.sprint3_gold.calculate_tax`
    (amount FLOAT64)
AS (amount * 1.21);
```

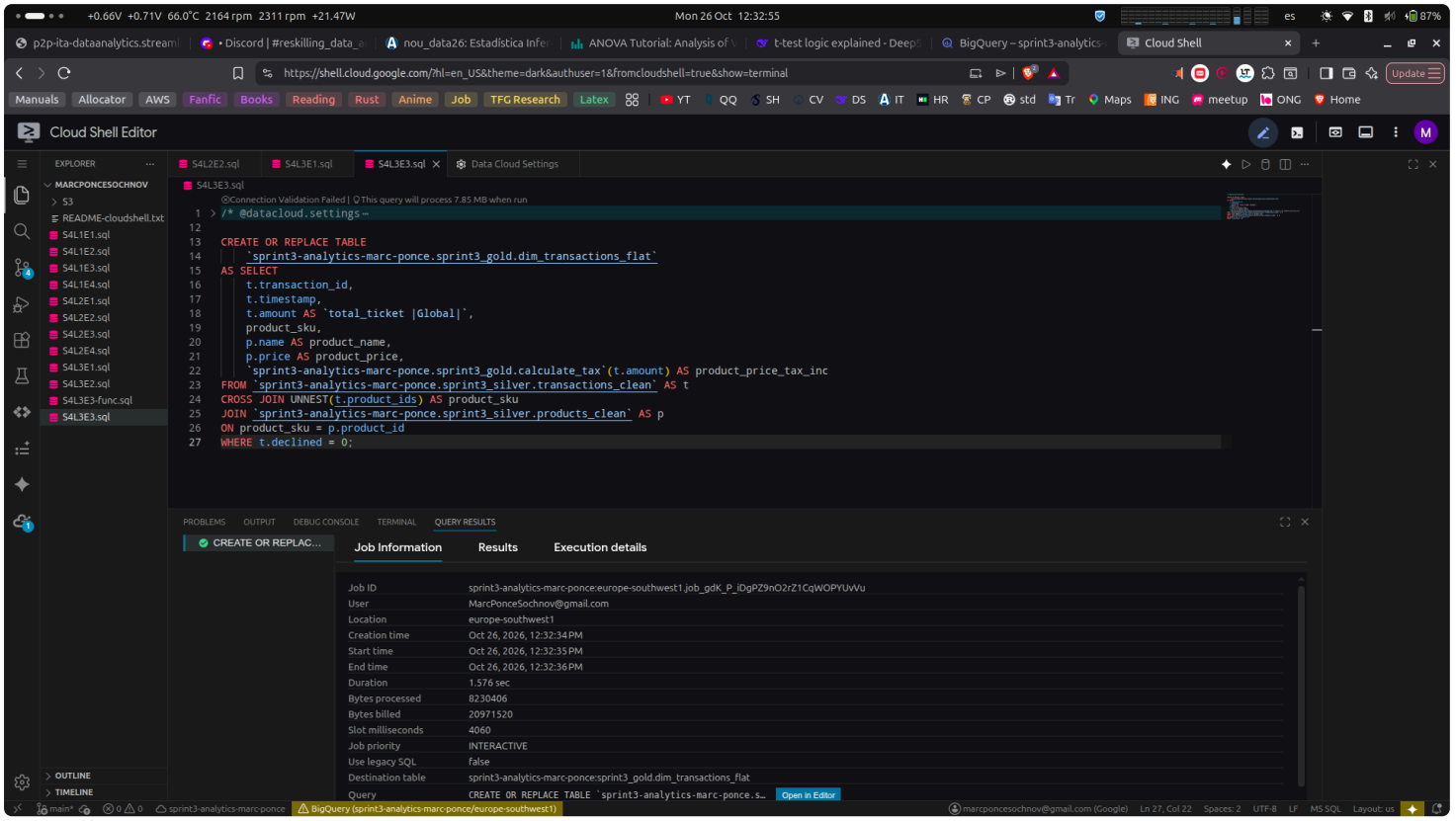


» No bytes billed or processed* in defining a function.

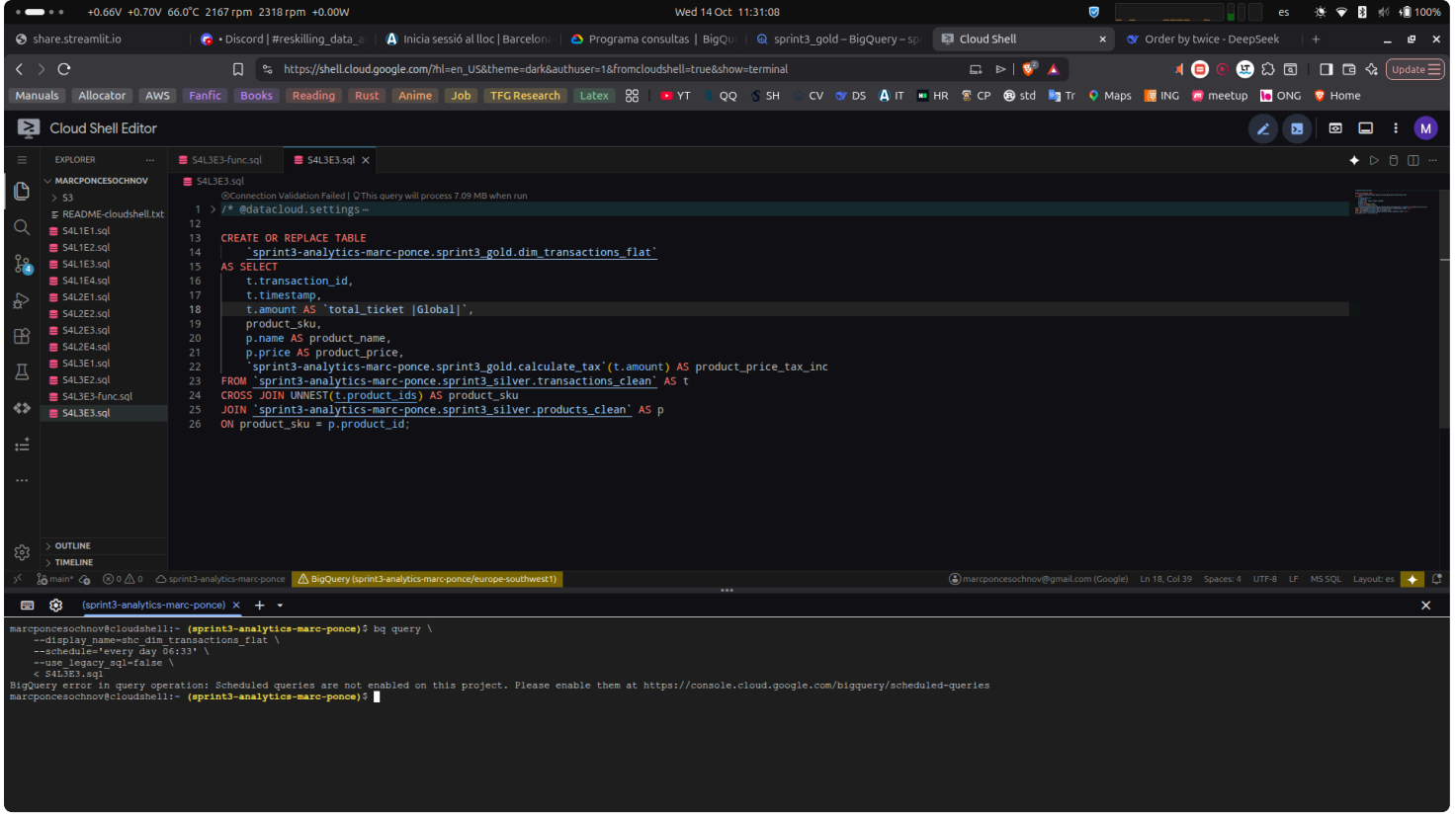
```
CREATE OR REPLACE TABLE
    `sprint3-analytics-marc-ponce.sprint3_gold.dim_transactions_flat`
AS SELECT
    t.transaction_id,
    t.timestamp,
    t.amount AS `total_ticket |Global|`,
    product_sku,
    p.name AS product_name,
    p.price AS product_price,
    `sprint3-analytics-marc-ponce.sprint3_gold.calculate_tax`(t.amount) AS
product_price_tax_inc
FROM `sprint3-analytics-marc-ponce.sprint3_silver.transactions_clean` AS t
CROSS JOIN UNNEST(t.product_ids) AS product_sku
JOIN `sprint3-analytics-marc-ponce.sprint3_silver.products_clean` AS p
ON product_sku = p.product_id
WHERE t.declined = 0;
```

» Previously I forgot to add the `declined = 0` condition, I will change just these two screenshots so that it can be seen that query is accepted.

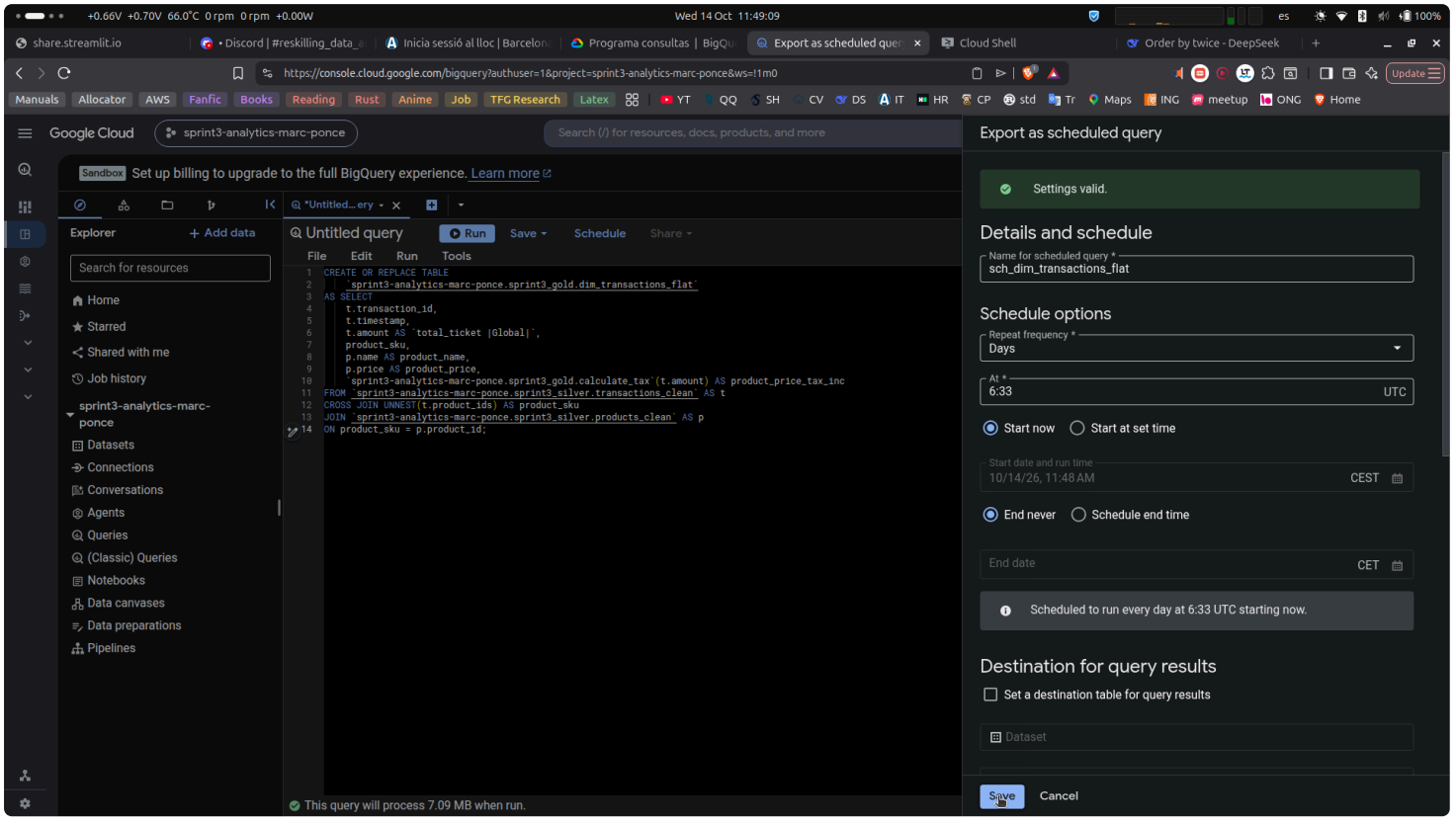


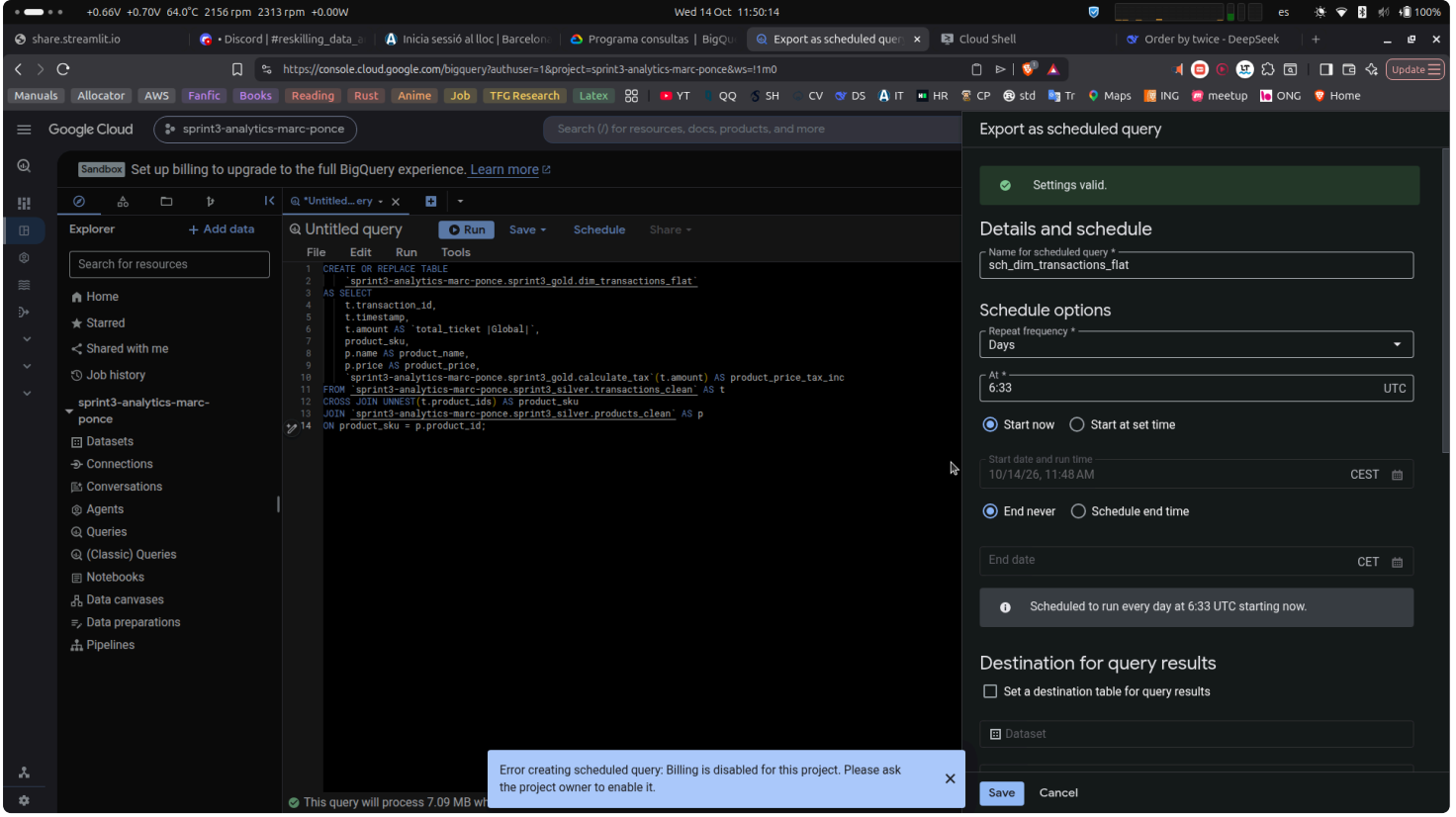



```
bq query \
    --display_name=shc_dim_transactions_flat \
    --schedule='every day 06:33' \
    --use_legacy_sql=false \
    < S4L3E3.sql
```



» I have gone through some steps with the Gemini AI assistant, and there seem to be some other roadblocks regarding my consent on the use of this service. Since there is no clear button that would let me enable the scheduling (not one I have found), I will proceed by doing the rest from the console.





» It seems I can not complete this exercise without enabling project billing.

Ingres per transacció.

📌 Proposta de Dashboard:

"Monitor de Rendiment de Vendes"

Configura una pàgina amb 3 seccions clau:

1. **Els "Big Numbers" (KPIs Generals)**
A la part superior, col·loca tres targetes de resultats (Scorecards) per donar context immediat.
- **KPI 1: Ingressos Totals (Amb IVA)**
- **Mètrica**: `SUM(product_price_tax_inc)`
- *Per què*: Mostra el volum real de diners que entra.
- **Transaccions Úniques (Comandes)**
- **Mètrica**: `COUNT_DISTINCT(transaction_id)`
- **Transaccions Úniques (Comandes)**
- **Camp Calculat**: Crea un camp a Looker: `SUM(product_price_tax_inc) / COUNT_DISTINCT(transaction_id)`

2. **Evolució Temporal (Gràfic de Sèries Temporals)**
Sota els KPIs, un gràfic de línia que ocupi tot l'ample.
- **Dimensió**: timestamp (Desglossat per Data).
- **Mètrica**: `SUM(product_price_tax_inc)` .
- **Línia de Referència (Opcional)**: Afegeix una mitjana constant o una línia de tendència.
- **Valor**: Permet a Màrqueting veure si les vendes pugen o baixen dia a dia, o si hi va haver un pic per alguna campanya específica.

3. **El "Top Productes" Enriquit (Taula amb Mapa de Calor)**
   Oblida't del gràfic de barres simple. Utilitza una Taula amb Barres incrustades o Mapa de Calor.
   - **Dimensió**: `product_name`.
   - **Mètrica 1**: `COUNT(*)` → Renombra-ho a "Unitats Venudes".
   - **Mètrica 2**: `SUM(product_price_tax_inc)` → Renombra-ho a "Ingressos Totals".
   - **Mètrica 3**: `SUM(product_price_tax_inc) - SUM(product_unit_price)` → Renombra-ho a "IVA Recaptat" (Això demostra el valor de la teva UDF).
   - **Estil**: A la configuració d'estil de la taula, activa l'opció "Mostra número com a: Barra" o "Mapa de calor" per a la columna d'Ingressos.

[Report Link]

